

MLOPS · CHURN PREDICTION · REAL DATA

Subscriber Dropout Detection

The complete project, end to end, and a viva question bank — every number in this book comes from the running system.

82.8M

events in a Postgres warehouse

8

services, one command

434

tests · 7 CI/CD jobs

Contents

How to use this book

PART I · THE PROJECT, END TO END

- 1 What this project is
 - 2 Architecture
 - 3 Technology stack, and why each piece
 - 4 A tour of the repository
 - 5 The data: from 31 GB of CSV to a warehouse
 - 6 Features without leakage
 - 7 Training and evaluating the model
 - 8 The registry and the promotion gate
 - 9 Serving: the API, explanations and the dashboard
 - 10 Streaming inference with Kafka
 - 11 Monitoring, drift and alerting
 - 12 Orchestration and reproducibility
 - 13 CI/CD: testing the system, not just the code
 - 14 Deployment: Docker, Compose and Kubernetes
 - 15 Running the demo
 - 16 Results, limitations and future work
 - 17 What building it taught
-

PART II · VIVA QUESTION BANK

- A · The project and its purpose
 - B · Data and the warehouse
 - C · Features and data leakage
-

D · The model

E · Metrics and evaluation

F · Thresholds, costs, calibration and fairness

G · Explainability

H · Registry, promotion and shadow scoring

I · Serving and the API

J · Streaming

K · Monitoring, drift and alerting

L · Orchestration and reproducibility

M · CI/CD and testing

N · Docker, Compose and Kubernetes

O · General ML and MLOps theory

P · Limitations, ethics and hard questions

APPENDICES

A · The whole project on one page

B · Numbers worth knowing

C · Commands

D · Glossary

How to use this book

Two halves. Part I explains the whole system, end to end, in the order the data flows. Part II is a question bank: the questions an examiner is likely to ask, with answers.

Read Part I once, slowly. Every chapter ends with a short "Say this" box: the two or three sentences worth being able to say without notes. Part II assumes you've read Part I; its answers are short on purpose.

Learn ideas, not wording. An examiner who hears a memorised paragraph asks a follow-up, and the follow-up is where understanding shows. Many answers in Part II include an *"If they push"* line: the most likely follow-up and how to handle it.

Every number is real. Figures come from the trained model's own reports (`src/models/artifacts/metrics.json`), the Postgres warehouse, the CI runs, or measurements taken while building the demo. Where a number is an assumption, such as the cost figures, the book says so.

BE HONEST ABOUT THE WEAK PARTS

The model is modest, one fairness check fails, and the cost figures are placeholders. Part I says so plainly, and so should you. Examiners respect a student who knows exactly where their system is weak far more than one who claims it has no weaknesses. Chapter 16 lists every limitation.

Tags used in Part II

CORE Very likely to be asked. Know these cold.

TRICKY Questions designed to find the edge of your understanding, or that touch a known weakness. Prepare these carefully.

PART I

The project, end to end

What the system does, how every piece works, and why each design decision was made, from 31 GB of raw CSV to a monitored model served in production.

CHAPTER 1

What this project is

A production-style machine learning system that predicts which subscribers of a music-streaming service are about to cancel, explains why, and keeps itself trustworthy after it ships.

The business problem

A subscription business earns money every month a customer stays. When a subscriber cancels ("churns" or "drops out"), that revenue is gone, and winning a customer back costs far more than keeping one. If the business can tell *in advance* who is likely to cancel, it can act while they're still a customer: a discount, a better-suited plan, a reminder of what they'd lose.

This project answers one question for every subscriber: **what is the probability they cancel in the next 30 days?** It also says *why*, and it runs the whole thing the way a real company would: versioned data, a gated model registry, an API, streaming, monitoring, alerting and automated testing.

What "MLOps" means here

Training a good model once is machine learning. **MLOps** is everything that keeps a model good in production:

- **Data you can trust:** validated on the way in, stored so the past can be rebuilt exactly.
- **Training you can reproduce:** versioned code, data and parameters.
- **A registry and a gate:** a record of every model, and a rule that stops a worse model reaching production.
- **Serving:** the model behind an API and a streaming consumer, with explanations.
- **Monitoring:** metrics, drift detection and alerts, so a model that is quietly getting worse is noticed.
- **Automation:** a scheduled retraining pipeline, and CI/CD that tests the *system*, not just the code.

The model itself (gradient boosting) is a small part of the codebase. Most of the work, and most of what is worth discussing in a viva, is everything around it.

The project in numbers

31.3 GB

raw KKBox CSV

82.8Mrows in the Postgres
warehouse**1.5%**churn rate in any 30-
day window**0.073**test PR-AUC (random =
0.015)

8

Docker services,
started by `make demo`

15

Prometheus alert rules

434

automated tests

7

CI/CD jobs on every
push

What was built, in one paragraph

The KKBox churn dataset (real, public, 31.3 GB) is validated, cleaned and loaded into an **event warehouse** in Postgres. A **point-in-time feature pipeline** builds training data for any past date without leaking the future. A **gradient-boosting model** is trained on a time-based split, audited for calibration, cost and fairness, and registered in **MLflow**. It only becomes the live `@champion` by beating the current one on **PR-AUC**. The champion is served by a **FastAPI** service with **SHAP** explanations and a dashboard, and by a **Kafka stream scorer** with a dead-letter queue. **Prometheus**, **Alertmanager** and **Grafana** watch it, with **PSI drift detection**. A **Prefect** pipeline retrains on a schedule, **DVC** versions the data pipeline, and **GitHub Actions** runs seven jobs on every push, including training on two databases and publishing the Docker image.

SAY THIS

"It predicts which subscribers will cancel in the next 30 days, on real data from a music-streaming service. But the model is the small part. The project is really about MLOps: making a model trustworthy in production, with a warehouse that can't leak the future, a registry with a promotion gate, explained predictions, drift monitoring, and CI that tests the whole system."

CHAPTER 2

Architecture

Four stages (data, model, serve, watch) with automation running underneath all of them.

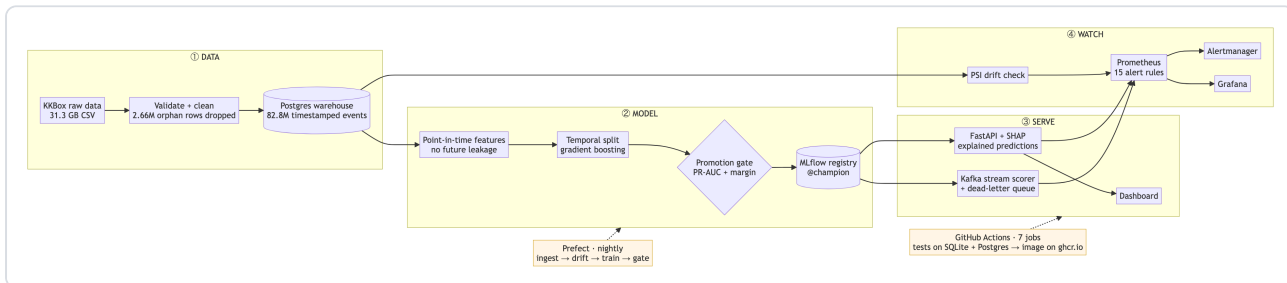


Figure 2.1: The system in four stages. Dashed arrows are automation.

The four stages

STAGE	WHAT HAPPENS	MAIN CODE
1 • Data	Raw KKBox CSV is validated against a contract, mapped to the warehouse schema, cleaned (orphans dropped, prices normalised) and stored as timestamped events.	<code>src/data/external/</code> , <code>src/warehouse/</code>
2 • Model	Point-in-time features for chosen cutoff dates; time-based split; gradient boosting; decision-quality audit; registered in MLflow and promoted only through the PR-AUC gate.	<code>src/features/</code> , <code>src/models/</code> , <code>src/registry/</code>
3 • Serve	The champion model answers HTTP requests (single, batch) with SHAP explanations, powers the dashboard, and scores a Kafka event stream.	<code>src/api/</code> , <code>src/streaming/</code>
4 • Watch	Metrics scraped by Prometheus, 15 alert rules routed by Alertmanager, a Grafana dashboard, and PSI drift checks against the training distribution.	<code>src/monitoring/</code> , <code>deploy/</code>
Automation	Prefect runs ingest, drift, train and gate on a schedule; DVC tracks the pipeline; GitHub Actions tests and publishes on every push.	<code>src/orchestration/</code> , <code>dvc.yaml</code> , <code>.github/</code>

Two flows through the system

The training flow (offline, nightly or on demand)

1. New events land in the warehouse (`subscribers` , `subscription_events` , `payments` , `sessions`).
2. The drift step compares recent data against the serving model's training distribution. It runs *before* retraining (Chapter 12 explains why).

3. For each cutoff date, the feature pipeline builds one row per subscriber using only data before the cutoff, and a label from the 30 days after.
4. The model trains on older cutoffs, is tuned on a later one, and is tested on the latest.
5. The run is logged to MLflow and registered as `@challenger`. If it beats `@champion` on PR-AUC by at least 0.005, the champion alias moves to it.
6. A run report is written; Prometheus exposes it; alerts fire if something needs a human.

The serving flow (online, per request)

1. A client sends a subscriber's nine features to `POST /predict` (or events arrive on the `subscriber-events` Kafka topic).
2. The service validates the input (types, ranges, and that the last activity isn't longer ago than tenure).
3. The champion model returns a probability; it's mapped to a risk band (low / medium / high) using the decision threshold.
4. SHAP computes each feature's contribution; the top reasons become a plain-language explanation.
5. If a distinct challenger exists, it scores the same request silently (shadow scoring).
6. Counters and histograms update; Prometheus scrapes them every 15 seconds.

SAY THIS

"Left to right: data, model, serve, watch. Raw data is cleaned into a Postgres event warehouse; point-in-time features train a model that only reaches production through a PR-AUC gate in MLflow; the champion is served by an API and a Kafka consumer; Prometheus and Grafana watch it. Prefect and GitHub Actions automate all of it."

CHAPTER 3

Technology stack, and why each piece

Every tool is free and open-source (or has a free tier) and was chosen for a specific job. Be ready to justify any row of this table.

LAYER	TOOL	WHY THIS ONE
Language	Python 3.11	The standard for ML; every library below is Python-native.
Warehouse	PostgreSQL 16 (SQLite as local fallback)	A real database server with parallel queries and a proper planner: the same feature query ran in 6.8 s on Postgres against 506 s on SQLite. SQLite keeps the project runnable with no server at all.
Database access	SQLAlchemy 2 + psycopg3	The schema and queries are written once and run on both databases; one setting (<code>SDD_DATABASE_URL</code>) switches.
Data processing	pandas, NumPy	Tabular data handling; date arithmetic is done here, not in SQL, so queries stay portable.
Model	scikit-learn gradient boosting	Strong on tabular data, handles non-linear effects, and works with SHAP's exact tree explainer.
Explanations	SHAP (TreeExplainer)	Exact per-prediction attributions that add up to the model's output: the explanation can't disagree with the score.
Experiment tracking + registry	MLflow 3	Records every run; the registry's aliases (<code>@champion</code> , <code>@challenger</code>) decide which model is live.
Serving	FastAPI + Uvicorn + Pydantic	Typed request validation, automatic API docs at <code>/docs</code> , and async serving.
Streaming	Redpanda (Kafka-compatible) + kafka-python	Kafka's protocol without the JVM and ZooKeeper; runs as one container.
Metrics	Prometheus + prometheus-client	The standard pull-based metrics system; alert rules live next to the metrics.
Alert routing	Alertmanager	Groups, routes by severity, and suppresses alerts that another alert already explains.
Dashboards	Grafana	Provisioned from a JSON file in the repo, so the dashboard is version-controlled.
Orchestration	Prefect 3	Turns pipeline steps into tasks with retries, logging and a run history; can schedule nightly.

LAYER	TOOL	WHY THIS ONE
Data versioning	DVC	Declares pipeline stages with inputs, parameters and outputs; reruns only what changed.
Containers	Docker + Docker Compose	Eight services start identically on any machine with one command.
Orchestrated deployment	Kubernetes manifests	Deployments with health probes, an autoscaler and a disruption budget: the step after Compose.
CI/CD	GitHub Actions + GitHub Container Registry	Free for public repos; seven jobs test the system, and the image is published on every merge to <code>main</code> .
Database browser	TablePlus (free version)	A clear window into the Postgres warehouse for the demo.
Quality	pytest, ruff	434 tests; ruff for linting and consistent style.

WHY REDPANDA INSTEAD OF APACHE KAFKA?

Redpanda speaks the Kafka protocol, so the code uses a standard Kafka client and would run unchanged against Apache Kafka. It's a single binary with no JVM and no ZooKeeper, which makes it light enough for a laptop and for CI.

CHAPTER 4

A tour of the repository

Where each part of the system lives. Useful when an examiner says "show me the code for that".

```

subscriber-dropout-detection/
├── src/
│   ├── config/settings.py      every setting, overridable by SDD_* environment
│   └── variables
│       ├── data/external/      KKBBox loader: contract.py (validation), kkbox.py
│       │   (mapping),
│       │   ├── warehouse/
│       │   │   helpers),
│       │   ├── features/
│       │   ├── models/
│       │   ├── evaluation/
│       │   ├── registry/
│       │   └── api/
│       (dashboard)
│       ├── streaming/
│       └── produce.py
│           ├── monitoring/
│           ├── orchestration/
│           └── demo.py
├── tests/
├── deploy/
│   ├── prometheus/
│   ├── alertmanager/
│   ├── grafana/
│   ├── kubernetes/
│   └── postgres/init/
├── docs/viva/
├── .github/workflows/ci.yml
├── docker-compose.yml
├── Dockerfile
├── dvc.yaml, params.yaml
├── constraints-model.txt
├── Makefile
└── DEMO_QUERIES_POSTGRES.sql

```

ingest.py (orphan removal, writing)
 schema.py (5 event tables), database.py (engine,
 simulate.py (synthetic data), to_postgres.py (bulk copy)
 point_in_time.py (leak-free snapshots),
 build_features.py (derived features, preprocessing)
 train.py (training entry point), costs.py (cost model,
 capacity), calibration.py, evaluate.py
 explain.py (SHAP), fairness.py (group audit)
 tracking.py (MLflow logging), promote.py (the gate)
 main.py (routes), schemas.py (validation), service.py
 (loading, prediction, shadow), static/index.html
 runner.py, processor.py (dead-lettering), kafka.py,
 drift.py (PSI), profile.py (reference profile),
 prometheus.py (metrics), tracker.py (rolling window)
 pipeline.py (steps), flows.py (Prefect flows)
 make demo / demo-check / demo-down
 434 tests in 15 files
 prometheus.yml (scrape targets), alerts.yml (15 rules)
 alertmanager.yml (routing, inhibition)
 provisioned datasource + dashboard JSON
 api.yaml, stream-scorer.yaml, config.yaml
 creates MLflow's own database
 architecture, demo script, questions, slides, this book
 7 CI/CD jobs
 8 services
 multi-stage image, non-root user
 pipeline stages and every tunable parameter
 model libraries pinned to the training versions
 every workflow as one command
 the eight demo queries

The most important files

FILE	WHY IT MATTERS
<code>src/features/point_in_time.py</code>	The leakage defence. If this is wrong, every metric the project reports is wrong.
<code>src/registry/promote.py</code>	The gate: the only way a model reaches production.
<code>src/api/service.py</code>	Model loading, prediction, risk bands, explanations and shadow scoring.
<code>src/monitoring/drift.py</code>	PSI drift detection.
<code>src/orchestration/flows.py</code>	The pipeline as Prefect flows.
<code>.github/workflows/ci.yml</code>	What "tested" means in this project.

CHAPTER 5

The data: from 31 GB of CSV to a warehouse

Real data from a real subscription business, cleaned with decisions that can each be defended, and stored so that any past date can be rebuilt exactly.

The source: KKBox

KKBox is a music-streaming service in Taiwan, the Asian equivalent of Spotify. Its churn dataset was released for the **WSDM Cup 2018** competition on Kaggle and is public. It's anonymised: every subscriber is identified by `msno`, a 44-character base64 hash. There are no names, emails or anything identifying.

FILE	WHAT IT CONTAINS	SIZE
<code>members.csv</code>	One row per subscriber: city, age, gender, registration channel (<code>registered_via</code>), registration date	6.77M rows
<code>transactions.csv</code>	Payments and plan changes: plan length in days, list price, amount paid, auto-renew flag, transaction date, expiry, cancel flag	21.5M rows
<code>user_logs.csv</code>	One row per subscriber per day of listening: songs played to 25/50/75/98.5/100%, unique songs, total seconds	392M rows

Together: **31.3 GB** of CSV. Downloading it was itself a small problem: newer Kaggle API tokens (the `KGAT_` kind) need "Bearer" authentication, but the installed Kaggle command-line tool only sent the older "Basic" kind and was rejected. The fix was a direct download that sent the token correctly.

Why a warehouse of events

The single most important design decision in the data layer: **store immutable, timestamped events, not one pre-summarised row per subscriber.**

A model is trained on the past. To build a training example "as of 1 November 2016", we need each subscriber exactly as they were *on that date*: their plan then, their sessions in October, their payments before then. A table of running totals ("total sessions: 412") can't answer that, because the totals already include everything that happened after November. A table of events can answer it for *any* date, simply by filtering on the timestamp.

The warehouse schema

Five tables, defined once with SQLAlchemy in `src/warehouse/schema.py`:

TABLE	ONE ROW PER	KEY COLUMNS	ROWS
subscribers	subscriber	subscriber_id (primary key), signup_date, acquisition_channel	6,769,473
subscription_events	lifecycle event	event_type (signup / renewal / cancellation), plan_type, monthly_fee, is_auto_renew_enabled, occurred_at	18,891,703
payments	payment	amount, status, discount_applied, occurred_at	18,891,703
sessions	day of listening	duration_minutes, occurred_at	38,216,556
support_tickets	ticket	category, occurred_at (KKBox has none)	0

Every event table has a composite index on (subscriber_id, occurred_at), because nearly every query asks "this subscriber's events before this date". That index is why looking up one subscriber's full history takes 17 milliseconds out of 82.8 million rows. Total: **82.8 million rows, about 16 GB** in Postgres.

Mapping KKBox onto the schema

DECISION	WHAT WAS DONE	WHY
Plan tier	From plan length: 0–31 days → basic, 32–180 → standard, 181+ → premium	KKBox has no tier names, only durations. The mapping is an assumption, stated openly.
Monthly fee	Price ÷ plan days × 30.44	Plans run 7 to 410 days; a 410-day payment and a 30-day payment aren't comparable prices.
Event type	First transaction = signup; is_cancel = cancellation; otherwise renewal	is_cancel is the actual lifecycle fact and wins over position.
Discount	Paid less than list price → discount_applied	The one inference the raw data supports directly.
Acquisition channel	registered_via 7 → via_7	It's an opaque code; inventing names like "organic search" would put a false claim in the data.
Session duration	Total seconds ÷ 60, negatives clipped to 0	Negative durations are a known KKBox logging artefact.
Payment status	Every payment "succeeded"	KKBox only records a transaction when money moved; there are no failures to find.

The cleaning decisions, and how to defend them

1 · 2,656,043 orphan rows dropped

The transactions file references **432,623 subscribers who aren't in the members file**: 18.3% of everyone who ever transacted. Two options: invent a subscriber row for each (using their first transaction as a signup date), or drop them.

They were **dropped**. The earliest transaction for these subscribers falls on the first day of the log, which means they almost certainly signed up *before* the data begins. Inventing a signup date would make long-standing, loyal customers look brand new, and the model reads "new" as risky. Losing rows is visible and countable; silently corrupting the tenure of 432,623 people is neither.

2 · Subscriber IDs widened from 32 to 64 characters

The schema originally declared IDs as `VARCHAR(32)`; real IDs are 44 characters. **SQLite ignored the limit silently. Postgres would have rejected every single row.** This is the first of several bugs in this project that SQLite hid and Postgres exposed.

3 · Listening history bounded: 392M → 38.2M rows

Only sessions from **1 October 2016 to 28 February 2017** were loaded. Features look at most 30 days before a cutoff (plus all-time "last seen"), and the cutoffs used run from November 2016 to February 2017. Loading the full 392 million rows would have taken over 7 hours for data no feature uses.

4 · Negative durations clipped; prices normalised

Both covered in the mapping table above. Dropping rows with negative durations would understate activity, which the model would read as churn risk. Clipping to zero keeps the row.

TWO FEATURES THAT LEARN NOTHING

`support_tickets_last_90d` and `payment_failures_last_6m` are always zero on KKBox: there's no customer-service data, and every payment succeeded. The model can't learn from a constant. They remain in the API because the request format requires them, and the dashboard says so openly. Be ready to explain this before you're asked.

Validation before loading: the data contract

`src/data/external/contract.py` checks every table before anything is written: required columns present, types parseable, values in range, IDs non-empty, and events referring to known subscribers. A `--dry-run` mode reads and validates everything and writes nothing. On a 30 GB export, that's the difference between finding a date-format surprise in ten minutes and finding it three hours into a half-finished load.

SQLite and Postgres: one schema, two databases

The code talks to the database through SQLAlchemy, so the same schema and queries run on both. One setting decides which: `SDD_DATABASE_URL`.

- **SQLite** is a single file (`src/data/warehouse.db` , 15 GB). The project runs with no database server at all, which suits development and tests.
- **Postgres 16** runs as a Docker container in the Compose stack. It's what the demo, the dashboard and TablePlus use.

The real data was first loaded into SQLite, then copied into Postgres by

`src/warehouse/to_postgres.py` (`make warehouse-postgres`). It streams each table out of SQLite as CSV straight into Postgres's bulk `COPY` command, in 5-million-row slices, with no temporary files. Secondary indexes are dropped during the load and rebuilt once afterwards, which is much faster than updating them on every insert. The whole copy took **about 7.5 minutes** (up to ~500,000 rows per second), and the row counts were verified to match SQLite **exactly** for every table.

7.5 min

to copy 82.8M rows into Postgres

17 ms

to fetch one subscriber's full
history

74×

faster feature query on Postgres
(6.8 s vs 506 s)

Where Postgres actually runs

Postgres isn't installed on the Mac itself. It runs inside **Docker**, as the `subscriber-warehouse` container from the official `postgres:16-alpine` image, defined in `docker-compose.yml`. Its port 5432 is published to the Mac, so any client (TablePlus, the pipeline, psql) connects to `localhost:5432` as if Postgres were installed normally. The data lives in a Docker **named volume** (`warehouse-data`), which survives restarts and `make demo-down`. MLflow's own tables live in a separate `mlflow` database on the same server, created by an init script, so the warehouse contains only warehouse tables.

SAY THIS

"The data is real: KKBox, 31 GB, 6.8 million subscribers. I stored it as timestamped events, not totals, so I can rebuild any subscriber as of any past date. The biggest cleaning decision was dropping 2.66 million orphan transactions rather than inventing signup dates, because invented dates would make loyal customers look new. It lives in Postgres, running in Docker: 82.8 million rows, copied and verified row for row."

CHAPTER 6

Features without leakage

The model must never see anything it wouldn't have at prediction time. This chapter explains how the pipeline guarantees that, and the one subtle bug the guarantee exposed.

What leakage is

Data leakage is when training data contains information that wouldn't be available when the model is really used. The classic churn example: a feature like "days active in the last 30 days", computed *today* for a subscriber whose label comes from last month. That feature includes the very period in which they did or didn't cancel. The model looks brilliant in testing, because it's partly reading the answer, and fails in production.

Point-in-time correctness

Every training example is built "as of" a **cutoff date T**, with two windows that never overlap:

WINDOW	COVERS	USED FOR
Observation	The 30 days before T (180 days for payments, 90 for tickets)	Features, computed only from events with <code>occurred_at < T</code>
Prediction horizon	The 30 days from T	The label: did they cancel with <code>T ≤ occurred_at < T + 30 days</code> ?

Nothing at or after T can reach a feature. A test in the suite (`test_events_after_the_cutoff_cannot_change_features`) adds future events and asserts that no feature changes. It's the most important test in the project, because if it failed, every metric would be meaningless.

The label: what "churn" means here

A subscriber churns if they have an **explicit cancellation event** (KKBox's `is_cancel` flag) in the 30 days after the cutoff. Subscribers who had *already* cancelled before the cutoff are excluded: they can't churn twice, and including them would fill the data with trivially-zero labels.

KNOW THIS DIFFERENCE

The Kaggle competition defined churn differently: no new subscription within 30 days of the current one expiring. That includes quiet lapses, not just explicit cancellations. This project's definition is stricter, which is one reason its churn rate is only about 1.5%. If an examiner knows the dataset, expect this question.

How the feature query works

One SQL query (`src/features/point_in_time.py`) builds every subscriber's row for a cutoff, as a chain of common table expressions:

STEP	WHAT IT COMPUTES
<code>base</code>	Subscribers who signed up before the cutoff (optionally capped to N, ordered by ID)
<code>latest_state</code>	Each subscriber's most recent lifecycle event before T: current plan, fee, auto-renew. Uses <code>ROW_NUMBER()</code> ordered by time, then event ID for ties
<code>session_window</code>	Sessions in the 30 days before T: count and average duration
<code>session_recency</code>	The last session <i>ever</i> before T. It must look across all history: a subscriber absent for 90 days has no rows in a 30-day window, and treating that as "no data" would erase the strongest signal
<code>payment_window</code>	Failed payments and discounts used in the 180 days before T
<code>ticket_window</code>	Support tickets in the 90 days before T

Dates are passed to the query as bound parameters, and date arithmetic (tenure, days since last activity) happens in pandas afterwards. That keeps the SQL identical on SQLite and Postgres, which spell date maths differently. Tenure is measured at the cutoff, not today. A subscriber with no sessions at all gets "last active" equal to their tenure, and "last active" is never allowed to exceed tenure.

The model's inputs

INPUT (API FIELD)	MEANING
<code>tenure_days</code>	Days since signup, as of the cutoff
<code>plan_type</code>	basic / standard / premium
<code>monthly_fee</code>	Current fee, normalised to a month
<code>avg_session_count_last_30d</code>	Listening days in the last 30 days
<code>last_activity_days_ago</code>	Days since last listened
<code>support_tickets_last_90d</code>	Always 0 on KKBox
<code>payment_failures_last_6m</code>	Always 0 on KKBox
<code>discounts_used_last_6m</code>	Payments made below list price
<code>is_auto_renew_enabled</code>	Auto-renew on or off

From these, `src/features/build_features.py` derives: `tenure_months`, `recency_ratio` (days inactive ÷ tenure), `engagement_recency_score` (sessions ÷ days inactive), `fee_per_session`, `discount_dependency` (discounts ÷ 6), `friction_score` (tickets + failures), per-month rates, and `is_dormant`. The plan type is one-hot encoded. The same code runs in training and serving, so the model always sees identically built features.

Splitting by time, never randomly

SPLIT	CUTOFF(S)	ROWS
Train	1 Nov 2016, 1 Dec 2016	66,704
Validation	31 Dec 2016	33,600
Test	29 Jan 2017	33,042

A random split would put rows from January into training and rows from November into testing: the model would learn from the future and be graded on the past. The test score would measure memory, not prediction. A time-based split is how the model is really used: trained on what has happened, judged on what happens next.

The bug the Postgres comparison found

After moving the warehouse to Postgres, the same training snapshot was built from both databases and compared row for row. **They differed by exactly one subscriber out of 1,104.**

The cause: SQLite stores dates as *text*. The query compared `signup_date < cutoff`, with the cutoff passed as a timestamp, so SQLite compared the strings `'2016-12-01' < '2016-12-01 00:00:00'`. As text, that's **true**, because a shorter string that matches the start of a longer one sorts first. So SQLite counted everyone who signed up *on* the cutoff day (5,850 people) as having signed up *before* it. Postgres compares real dates and correctly excluded them.

Impact: no future information reached any feature, because those subscribers have no event strictly before the cutoff and a later join dropped them. But they occupied slots in the capped sample, displacing real subscribers. **Fix:** compare the date column against a real date. A regression test was written first, confirmed to fail, and passes after the fix. After it, the two databases produced **identical** snapshots: every row, feature and label.

SAY THIS

"For a cutoff date T, features only use the 30 days before T and the label only looks at the 30 days after. The windows never overlap, so the model can't see the answer. I split by time: train on November and December 2016, test on January 2017. And comparing SQLite with Postgres caught a real bug: SQLite compared dates as text and let same-day signups through. I fixed it with a test that failed first."

CHAPTER 7

Training and evaluating the model

A deliberately simple, well-understood model, judged with metrics that suit a rare event, and audited for whether it is actually fit for the decision it drives.

The model

A scikit-learn **gradient boosting classifier**: an ensemble of shallow decision trees, each built to correct the errors of the ones before it.

PARAMETER	VALUE	WHY
n_estimators	300	Enough trees to learn the signal at a low learning rate
learning_rate	0.05	Small steps generalise better than large ones
max_depth	3	Shallow trees capture interactions without memorising
subsample	0.9	Each tree sees 90% of rows, which adds randomness and reduces overfitting
min_samples_leaf	20	No leaf is built on a handful of subscribers

Why gradient boosting? It's among the strongest methods for tabular data with mixed numeric and categorical columns. It captures non-linear effects (risk that rises sharply after a few inactive weeks) without hand-built features. And it works with SHAP's exact *tree* explainer, so every prediction can be explained exactly. All parameters live in `params.yaml`, so changing one is a versioned, reviewable change.

Why accuracy is useless here

Only about **1.5%** of subscribers cancel in a 30-day window. A "model" that predicts *nobody ever churns* is right 98.5% of the time, and completely useless. This model's accuracy is 95.6%, *lower* than that useless baseline, because it deliberately flags people. Accuracy rewards ignoring the rare class, so it's never used to judge anything in this project.

PR-AUC, ROC-AUC, and the finding that shaped the project

ROC-AUC measures how well the model ranks a random churner above a random non-churner. It uses the false-positive *rate*, which divides by the huge number of non-churners, so it barely notices how rare churn is. **PR-AUC** (area under the precision–recall curve) is built from precision: "of the people I flagged, how many really churn?". That is dominated by the rare class. A random model's PR-AUC equals the churn rate.

The project saw the difference directly. Moving from synthetic data (about 20% churn) to real KKBox data (1.5% churn):

METRIC	SYNTHETIC DATA	REAL KKBOX	DIRECTION
ROC-AUC	0.674	0.835	Up
PR-AUC	0.345	0.073	Down, sharply

ROC-AUC said the model got *better*. PR-AUC said the problem got much *harder*, and PR-AUC is the one that matches the business reality: with rarer churners, far more of the people you flag are false alarms. **If the promotion gate had used ROC-AUC, it would have reported an improvement at the exact moment the model became harder to act on.** That's why the gate uses PR-AUC. The decision was made before the real data arrived, and the real data confirmed it.

Results on the held-out test set (29 January 2017)

METRIC	VALUE	WHAT IT MEANS
PR-AUC	0.073	About 4.8× better than random (0.015)
ROC-AUC	0.835	Ranks a churner above a non-churner 83.5% of the time
Churn rate	1.53%	506 churners among 33,042 subscribers
Recall @ 0.05	16.8%	Catches 85 of 506 churners: about 1 in 6
Precision @ 0.05	7.6%	1 in 13 flagged subscribers really churns: 5× the base rate
Flagged	3.4%	1,125 of 33,042 subscribers
Accuracy	95.6%	Shown only to prove the point above
Calibration error (ECE)	0.004	Predicted probabilities match observed rates closely

At the 0.05 threshold the confusion matrix is: **85** true positives, **1,040** false positives, **421** false negatives, **31,496** true negatives.

Which features matter most

FEATURE	IMPORTANCE
is_auto_renew_enabled	0.283
monthly_fee	0.167
tenure_days	0.097
recency_ratio	0.096

FEATURE	IMPORTANCE
<code>discount_dependency</code>	0.076
<code>tenure_months</code>	0.071

THE SURPRISING ONE: AUTO-RENEW RAISES RISK

Intuition says auto-renew should *lower* churn risk. On KKBox it raises it: on an otherwise identical subscriber, turning auto-renew on moves the prediction from 0.0024 to 0.6037. In this data, auto-renew goes with the cheap, short plans that churn most. The model learned a **real association in this data, not a cause**. That's the honest answer, and a good example of why models need explanations and human judgement.

Choosing the decision threshold

The model outputs a probability; a threshold turns it into a decision (flag or not). The default 0.5 would flag almost no one when churn is 1.5%. The served threshold is **0.05**. It's the value that **maximised F1 on the validation set**, searched over 0.05 to 0.95 in steps of 0.01. It sits at the very bottom of that range, which means F1 would have preferred it even lower.

The decision-quality audit

Every training run writes a report asking whether the model is fit for the decision it drives, not just whether its metrics look good. Four parts.

1 • Cost

Each outcome is given a cost (in `params.yaml`; these are **placeholder assumptions**, not real business figures):

Missed churner (false negative)	£240: about a year of lost revenue
Retention offer	£20
Offer efficacy	30%: an offer saves 3 in 10 real churners
Flagged churner (true positive)	$£20 + 70\% \times £240 = £188$
Flagged non-churner (false positive)	£20: the wasted offer

Flagging someone is worth it only when the expected saving beats the offer's cost: $p \times (240 - 188) > (1 - p) \times 20$, which gives $p > 0.278$. Searching thresholds on the test set, the cheapest is **0.43**: total cost £121,312, against £137,820 at the served 0.05, so **£16,508 cheaper**. At 0.43 the model sends only 8 offers out of 33,042.

THE HONEST READING

At a 1.5% churn rate, and with an offer that only works 30% of the time, almost no one is risky enough to be worth an offer. So the cost-optimal policy is "send almost nothing". F1 chose 0.05; the cost audit prefers 0.43. **The audit reports this rather than silently changing the threshold**, because the right answer depends on real costs the project doesn't have. Grafana shows it as "savings left on the table".

2 · Calibration

A model is **calibrated** when a predicted 5% really happens about 5% of the time. That matters whenever a probability drives a money decision. Expected calibration error is **0.004**, and the Brier score is 0.0146, both very good. On average it predicts 1.16% against an actual 1.53%, so it slightly under-predicts. Isotonic calibration is tried as a diagnostic on every run: fitted on the validation set, it would improve the error by a further 0.003. It's **reported, not applied**; the raw model is already well calibrated.

3 · Fairness

The audit compares performance across `plan_type` groups (groups need at least 30 members, and a ratio below 0.8 between the worst and best group is flagged: the US "four-fifths rule" from employment law, a convention rather than a law of nature). **It fails**. On the `standard` plan (949 subscribers in the test set) the model's ROC-AUC is 0.54, close to random, against much higher on the other plans. The selection rates also differ widely. The likely cause is too few standard-plan subscribers to learn from. The failure is written to the report, exported to Prometheus, shown as "DISPARITY" in Grafana, and fires the `FairnessDisparity` alert. **It's reported, not hidden**.

4 · Capacity

A real retention team can only contact so many people. The cost model supports a capacity cap (`max_offer_rate`): if the cost-optimal policy needs more offers than the budget allows, the threshold rises to the top-k subscribers and the report prices the shortfall. No cap is set for KKBox, since the cost-optimal policy only needs 8 offers.

SAY THIS

"Accuracy is useless at 1.5% churn: predicting 'nobody churns' is 98.5% accurate. I use PR-AUC, and the real data proved why: ROC-AUC went up while PR-AUC fell. The model gets 0.073, about 5× better than random. It's modest, and I say so. Every run is audited: it's well calibrated, the cost audit prefers a stricter threshold under placeholder costs, and the fairness audit fails on one plan, which is reported and alerted on."

CHAPTER 8

The registry and the promotion gate

No model reaches production by accident. Every model is recorded, and the only way to become the live model is to beat the current one on evidence.

MLflow: tracking and registry

Every training run is logged to **MLflow**: its parameters, metrics and the model itself. Models are registered under one name, `subscriber-dropout-classifier`, as numbered versions. Two **aliases** say which version plays which role:

- **@champion**: the version serving production traffic.
- **@challenger**: the newest candidate.

Serving loads "whatever `@champion` points to". Promoting a model means moving an alias; **rolling back means moving it back**. No files are copied and nothing is redeployed. (MLflow 3 removed the older "Staging/Production" stages altogether; aliases replace them.)

The gate

`src/registry/promote.py` decides. It has three properties:

1. **Comparable**: champion and challenger are scored on the *same* held-out data, in the same process. Comparing a challenger's fresh test score with a champion's score from months ago measures two different datasets, and that's a reliable way to promote a worse model.
2. **The right metric**: PR-AUC, for the reasons in Chapter 7.
3. **A margin**: the challenger must win by at least **0.005**. Two equivalent models differ only by noise; with a zero margin, the gate would promote on that noise about half the time, and the registry would flip back and forth forever.

If there's no champion yet, the first model is promoted unopposed. Every decision (promoted or rejected, with both scores and the required margin) is recorded in the run report.

PROVED IN CI, NOT JUST CLAIMED

The pipeline job in CI trains twice on identical data. The second model is identical to the champion, so it **must be rejected**, and CI fails if it isn't. A rejected challenger isn't a failure: it's the gate doing its job, so it raises an `info` alert, never a page.

Shadow scoring: judging a challenger on real traffic

Test-set scores are one thing; live behaviour is another. When a *distinct* challenger is registered, the API scores every request with **both** models but returns **only the champion's** answer. The challenger's answer is recorded and compared:

- **Agreement rate:** how often both models make the same flag / don't-flag decision.
- **Outreach volume change:** how many more or fewer people the challenger would flag, which tells the business how the campaign would change on promotion.
- **Divergence:** the typical difference between the two probabilities.
- **Errors and latency:** a challenger that crashes or is slow is caught before it matters.

Shadow scoring never affects the response. If the challenger fails, the champion's answer still goes out and the error is counted. A verdict needs at least 200 comparisons. Right now no distinct challenger exists, so the dashboard honestly shows "IDLE" and "no challenger".

SAY THIS

"Every run is logged in MLflow. The live model is whichever version holds the @champion alias. A new model only takes over if it beats the champion on PR-AUC, on the same data, by at least 0.005, so noise can't flip it. Rollback is just moving the alias back. CI proves an identical challenger gets rejected. And shadow scoring lets a challenger be judged on live traffic without ever answering a real request."

CHAPTER 9

Serving: the API, explanations and the dashboard

The champion model behind an HTTP API that validates every input, explains every answer, and degrades gracefully instead of failing.

The API

Built with **FastAPI** (`src/api/main.py`), served by Uvicorn on port 8000. Interactive documentation is generated automatically at `/docs` .

ENDPOINT	PURPOSE
<code>GET /</code>	The dashboard (a single self-contained HTML page)
<code>GET /health</code>	Liveness: is the process up? Always cheap; used by Docker and Kubernetes
<code>GET /ready</code>	Readiness: is a model loaded and able to serve?
<code>GET /model-info</code>	Which model is serving: name, training date, threshold, required inputs, library versions, source
<code>POST /predict</code>	Score one subscriber, with an explanation
<code>POST /predict/batch</code>	Score 1 to 1,000 subscribers in one call
<code>GET /metrics</code>	A JSON summary of recent traffic: counts, flagged rate, score percentiles, shift from training
<code>GET /metrics/prometheus</code>	The same, in Prometheus's text format, for scraping
<code>POST /monitoring/drift</code>	Compare a sample (up to 50,000 rows) against the training distribution with PSI
<code>GET /monitoring/shadow</code>	The champion-vs-challenger shadow report

Liveness vs readiness

Two different questions. **Liveness** (`/health`): is the process running? If not, restart it. **Readiness** (`/ready`): can it serve right now? If not, stop sending it traffic, but don't kill it. A service with no model is alive but not ready. It answers `/health` , and `/predict` returns **503 Service Unavailable** with a clear message rather than crashing with a 500. Kubernetes uses exactly these two endpoints for its probes.

Input validation

Every request is validated by **Pydantic** before it reaches the model: required fields present, correct types, values in range (for example, fees 0–10,000), plan type normalised. There's also a cross-field rule: **last activity can't be longer ago than the subscription's tenure**. Invalid input gets a **422** response naming the field and the problem, never a wrong prediction.

What a prediction returns

```
{
  "dropout_probability": 0.6037,
  "predicted_label": 1,
  "risk_level": "high",
  "threshold": 0.05,
  "explanation": "High dropout risk (60%): relies on discounts (2 in the last 6 months),
  ...",
  "top_risk_factors": ["relies on discounts ...", "auto-renew enabled", ...],
  "explanation_method": "shap",
  "attributions": [ { "feature": "discounts", "contribution": 3.062,
                      "direction": "increases_risk", "description": "...", ... } ]
}
```

Risk bands

low	below the decision threshold (5%): not flagged
medium	from 5% up to halfway between the threshold and 100%, about 52.5%
high	above that

Explanations with SHAP

SHAP (SHapley Additive exPlanations) comes from cooperative game theory. It asks how much each feature contributed to *this* prediction, compared with the average prediction. For tree models, SHAP's TreeExplainer computes the values **exactly**, not by approximation.

The key property is **additivity**: the average prediction plus every feature's contribution equals exactly what the model predicted (in log-odds, the scale the model works in internally). A test in the suite checks this for every row. So the explanation **can't disagree with the score**, unlike a rule-based explanation that describes the input without knowing what the model did.

Raw SHAP values are per model column, and some concepts span several columns (e.g. `tenure_days` and `tenure_months`). The service **sums them into ten concepts**: recency, engagement, tenure, payments, support, friction, discounts, price, auto-renew and plan. Summing is valid because Shapley values are additive. The strongest concepts, by absolute size, become the explanation. The wording states the *actual* input: an earlier version said "auto-renew enabled" for a subscriber who had it *off*, because it followed the direction of the effect rather than the value. That's fixed and tested.

If SHAP isn't available or fails, the prediction is still returned, with a rule-based explanation and `"explanation_method": "rules"`, so a client always knows which kind it got. **An explanation is a feature of a prediction, never a precondition for one.**

The dashboard

A single self-contained page (`src/api/static/index.html`, no build step) with four tabs, each reachable by link (`#score`, `#live`, `#model`, `#monitoring`):

TAB	WHAT IT SHOWS
Score	A form (sliders and inputs, with three presets), the probability, risk band, distance from the threshold, the SHAP drivers, and "What would change it"
Live traffic	Predictions served, flagged rate, risk mix, score spread, and live traffic against the training reference
Model	Provenance, library versions, required inputs, and notes on the two surprising behaviours
Monitoring	The shadow-scoring status, and a live drift test (below)

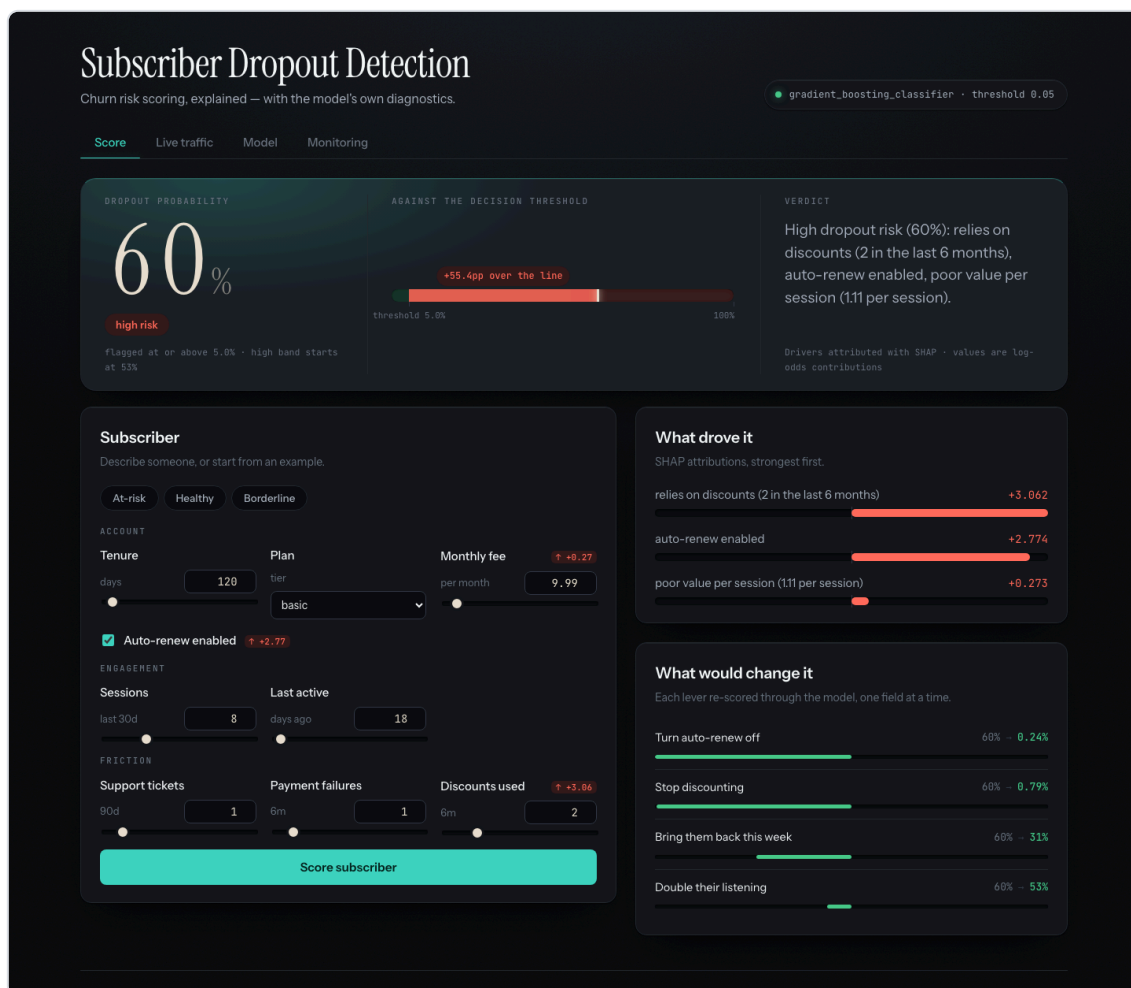


Figure 9.1: The Score tab, on the "at-risk" preset: 60%, high risk, with SHAP drivers and counterfactual levers.

The three presets

PRESET	PROBABILITY	BAND
At-risk	60.37%	high
Borderline	5.01%	medium (a hair over the threshold)
Healthy	0.05%	low

"What would change it": counterfactual levers

SHAP says what the model leaned on. It doesn't say what a retention team could *do*. So the dashboard sends real second requests to the model, each with **one field changed**, and shows the new probability. For the at-risk preset:

LEVER	60.4% BECOMES
Turn auto-renew off	0.24%
Stop discounting	0.79%
Bring them back this week	31%
Double their listening	53%

These are the model's own answers, not inferences. The two fields that are always zero in KKBox (tickets, payment failures) are deliberately never offered as levers, because they'd promise an action that changes nothing.

The live drift test

The Monitoring tab builds 400 plausible subscribers from the presets, then a second "disengaged" copy: sessions cut to near zero and last activity pushed back 45 days. It sends both to `/monitoring/drift` and compares. The detector ranks **exactly the two features that were changed** at the top, while the other seven move by less than 0.5. It's an instrument test: proof the detector responds to a change we control.

SAY THIS

"The API validates every input and returns a probability, a risk band and the reasons. The reasons are SHAP values, which add up exactly to the prediction, so the explanation can't contradict the score. The dashboard also asks the model what would change the outcome: each lever is a real second prediction with one field changed. And if the model or the explainer fails, the service degrades: a 503 or a rule-based explanation, never a crash."

CHAPTER 10

Streaming inference with Kafka

Scoring events as they happen, without losing data and without letting one bad message stop the pipeline.

Why streaming as well as an API

The API answers when asked. But subscriber events (a payment, a plan change, a quiet week) happen continuously, and the business wants a fresh risk score when they do, not only when someone remembers to ask. A stream scorer consumes events as they arrive and publishes scores for other systems to act on.

The setup

PIECE	VALUE
Broker	Redpanda (Kafka-compatible), one container, port 19092
Input topic	<code>subscriber-events</code>
Output topic	<code>subscriber-scores</code>
Dead-letter topic	<code>subscriber-scores-dlq</code>
Consumer group	<code>subscriber-scorer</code>
Batch size	up to 200 messages per poll
Metrics	Prometheus endpoint on port 8001

The consume → score → produce loop

1. Poll a batch of up to 200 messages from `subscriber-events`.
2. Parse and validate each one with the same schema the API uses.
3. Score the valid ones in a single batch call to the same prediction code the API uses.
4. Publish each score to `subscriber-scores`, keyed by subscriber (so one subscriber's scores stay in order in one partition).
5. Send each invalid message to `subscriber-scores-dlq`, with the **reason** and the **original payload** kept verbatim.
6. Wait for the broker to confirm the writes (`acks=all`), and **only then** commit the input offsets.

Delivery guarantee: at-least-once

Offsets are committed **only after** the scores are safely written. If the scorer crashes between producing and committing, it re-reads those messages on restart and scores them again: a duplicate, never a loss. That's **at-least-once** delivery. Auto-commit is switched off on purpose: it would acknowledge messages whose scores never reached the output topic, which is the one way this design could lose data. Duplicates are harmless here, because re-scoring the same subscriber gives the same score.

The dead-letter queue

A **poison message** (malformed JSON, a missing field, an impossible value) would crash a naive consumer. Worse, the consumer would restart, read the same message again and crash again: the classic **poison-pill loop** that takes a pipeline down for good. Here a bad message goes to the dead-letter topic with the reason attached, and the batch carries on. Every message ends up in exactly one place: scored, or dead-lettered with a reason.

One exception is deliberate. If the *model* itself is unavailable, messages are **not** dead-lettered, because that would throw away perfectly good data over a temporary problem. The batch fails, offsets aren't committed, and the messages are retried.

CI proves all of this: the streaming job sends deliberately broken messages and checks they land in the dead-letter topic while the good ones are scored.

SAY THIS

"The stream scorer reads events from Kafka, scores them, and writes the scores back. Offsets are committed only after the scores are safely written, so it's at-least-once: a crash causes a duplicate, never a loss. A malformed message goes to a dead-letter topic with the reason, so one bad message can't take the pipeline down. CI sends poison messages to prove it."

CHAPTER 11

Monitoring, drift and alerting

A model that was good at launch can quietly get worse. This chapter covers how the system notices.

Why models need monitoring

Ordinary software fails loudly: it crashes or returns errors. A model fails **quietly**. It keeps returning confident probabilities while the world it was trained on changes underneath it: prices change, a competitor launches, listening habits shift. Nothing errors. The predictions just get steadily worse. Monitoring exists to make that visible.

The monitoring stack

COMPONENT	ROLE
prometheus-client	The API and the stream scorer expose counters, histograms and gauges
Prometheus	<i>Pulls</i> (scrapes) those metrics every 15 seconds, stores the history, and evaluates the alert rules
Alertmanager	Receives firing alerts, groups them, routes them by severity, and suppresses redundant ones
Grafana	The dashboard, provisioned from a JSON file in the repository

Prometheus **pulls** rather than having services push. The monitoring system decides what and how often to collect, and a service that stops answering is itself a signal: its `up` metric drops to 0.

What is measured

- **Traffic:** predictions per second by risk band and by component (API or stream), and the score distribution (median, p90, p99).
- **Serving state:** model loaded, which source it was served from, the decision threshold, flagged rate, and the shift of the live mean score from the training mean.
- **Pipeline:** time since the last run, whether it needs attention, the last promotion decision, the drift verdict, and PSI per feature.
- **Decision quality:** calibration error, fairness pass/fail and ratios, savings available, and capacity.
- **Shadow scoring:** active or not, agreement rate, outreach change, divergence, latency and errors.

Drift detection with PSI

The **Population Stability Index** compares two distributions of one feature: training ("expected") and live ("actual").

1. Split the training values into 10 bins (by quantile, so each holds about 10% of training data). The bin edges are saved in `reference_profile.json` when the model trains.
2. Put the live values into the *same* bins.
3. For each bin, compare the live share a with the training share e , then add up the terms:

$$\text{PSI} = \sum (a - e) \times \ln(a / e)$$

(Shares are clipped at a tiny positive number first, so an empty bin can't cause a division by zero.) If the distributions are identical, every term is zero. The more the live data has moved, the larger PSI grows. Categorical features (plan type) use their categories as bins. The prediction itself is also checked, so a shift in the model's *output* is caught even when no single input moved much.

PSI	VERDICT	MEANING
< 0.10	stable	No meaningful change
0.10 – 0.25	moderate	Worth watching
> 0.25	significant	The population has shifted; consider retraining

Samples under 100 rows are flagged as too small to trust. PSI is noisy on tiny samples, and the response says so rather than reporting a confident verdict.

The real result

The pipeline's drift step was run on the **latest real month (February 2017)**, read from Postgres, against the training distribution (November–December 2016): 7,614 subscribers. Verdict: **stable**. Every feature's PSI was below 0.06: last activity 0.058, tenure 0.032, sessions 0.007, fee 0.003, plan 0.002. That's a real finding: KKBBox's subscriber base didn't shift in those months. The dashboard's drift test is how you show the detector *does* fire when something changes.

The alert rules

Fifteen rules in `deploy/prometheus/alerts.yml`. The guiding principle: **alert on what a human would actually act on**. Paging someone for trivia teaches them to ignore the pager.

ALERT	SEVERITY	FIRES WHEN
ModelNotLoaded	critical	The API is running without a model (5 min)
ApiDown	critical	Prometheus can't scrape the API (2 min)

ALERT	SEVERITY	FIRES WHEN
StreamScorerDown	critical	The stream scorer stops answering (5 min)
AlertmanagerDown	critical	The alert router itself is down (5 min)
FlaggedRateSpike	warning	More than half of scored subscribers are flagged (15 min)
ScoreDistributionShift	warning	Live mean score differs from training by over 0.15 (30 min)
SignificantDrift	warning	The last drift check was "significant" (10 min)
PipelineStale	warning	No pipeline run in over 36 hours
FairnessDisparity	warning	The fairness audit failed (30 min). Firing now, correctly.
ShadowChallengerErroring	warning	The challenger failed to score live traffic
PipelineNeedsAttention	info	A challenger was rejected, or drift/fairness flagged (1 h)
RetentionCapacityBinding	info	The outreach budget, not the model, limits retention (6 h)
OutreachVolumeUnaffordable	info	The cost-optimal policy implies a huge campaign (6 h)
ShadowLargeBehaviourChange	info	Champion and challenger disagree on most decisions (30 min)
ShadowOutreachVolumeShift	info	Promoting would change outreach volume sharply (30 min)

The duration in brackets is how long the condition must hold before the alert fires. It filters out brief blips.

Alertmanager: routing and inhibition

- **Routing by severity:** *critical* goes to an on-call receiver immediately and repeats hourly; *warning* goes to a review receiver and repeats every 12 hours; *info* is recorded and repeats daily.
- **Grouping:** alerts are grouped by name and component, so ten related alerts become one notification.
- **Inhibition:** if the API is down, warnings about it are suppressed, since they're symptoms of the same outage. If no model is loaded, alerts about the score distribution are suppressed.
- **No credentials in the repo:** the receivers ship empty. A test fails the build if anything credential-shaped (webhook URLs, routing keys) appears in the config.

Grafana

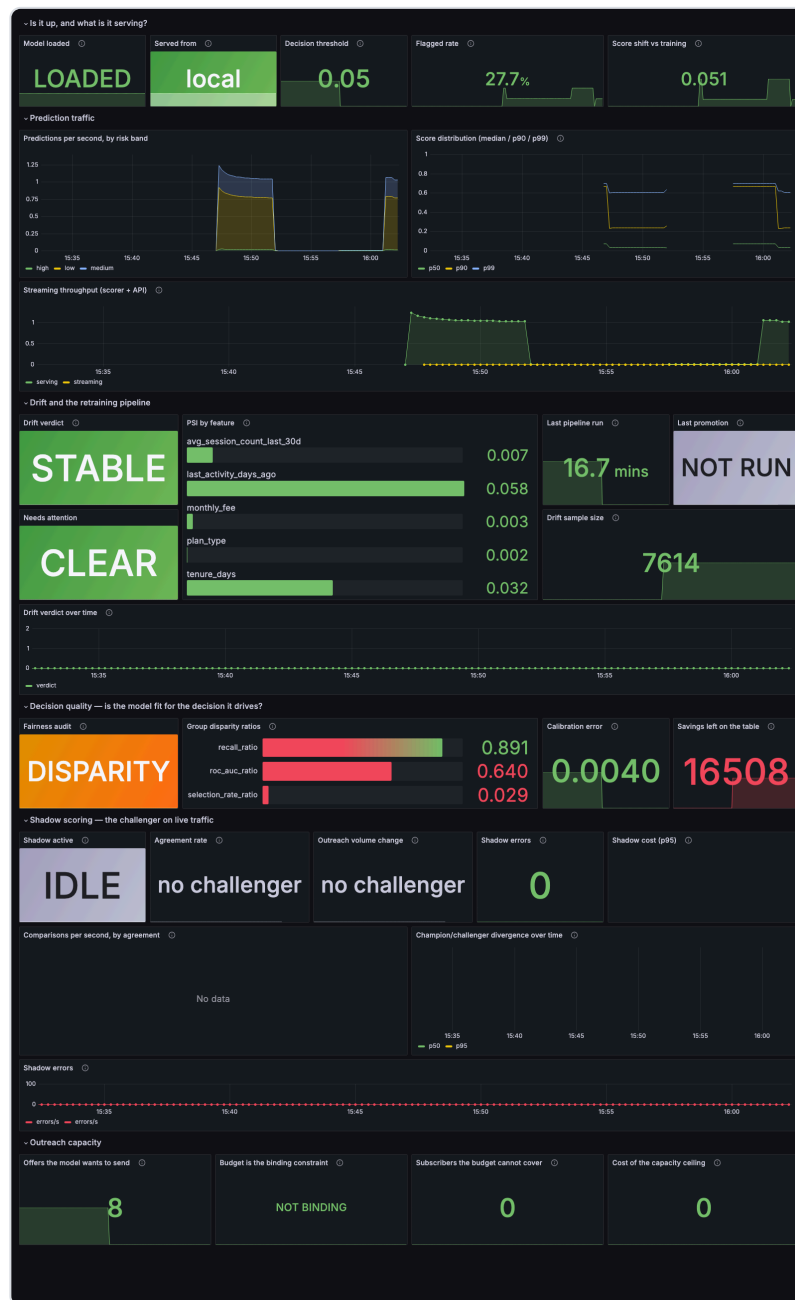


Figure 11.1: The Grafana dashboard. From the top: serving state, traffic, drift and the pipeline, decision quality, shadow scoring, capacity.

What looking at it found

With every automated check passing, a screenshot of Grafana showed that things were wrong:

- **Two values in every panel** ("LOADED | NO MODEL"). The stream scorer shares the API's metric registry but never updates the API's state gauges, so it exported their defaults. Worse, the **alert rules matched those zeros**: *PipelineStale* was firing and *ModelNotLoaded* was about to, on a healthy system. Fix: every API-state query is scoped to `job="subscriber-api"`, and a test fails if one isn't.

- **"Last promotion: REJECTED"** for a run that trained nothing, and **"0% agreement"** with no challenger. A gauge's default 0 is a *claim*. These now start as "no value" (NaN) and read "NOT RUN" and "no challenger".
- **Two sections drawn on top of each other**, plus a duplicated section. Re-laid out, with a test that no two panels overlap.

SAY THIS

"Models fail quietly, so the system watches them. Prometheus scrapes metrics every 15 seconds and evaluates 15 alert rules; Alertmanager routes them by severity and suppresses symptoms of the same outage. Drift is measured with PSI against the training distribution: on real February 2017 data every feature is under 0.06, so stable. And the one alert firing, fairness, is real."

CHAPTER 12

Orchestration and reproducibility

Running the pipeline on a schedule, safely re-runnable, and making any result traceable back to the exact data, code and settings that produced it.

The retraining pipeline

`src/orchestration/pipeline.py` holds the steps; `src/orchestration/flows.py` wraps them as **Prefect** tasks and flows.

STEP	WHAT IT DOES	RETRIES
1 • Ingest	Bring new events into the warehouse (or leave a populated warehouse alone)	2
2 • Assess drift	Compare recent data with the serving model's training distribution	–
3 • Train and gate	Train a challenger, register it, promote only if it beats the champion	1
4 • Report	Write <code>last_pipeline_run.json</code> , read by Prometheus and alerting	–

Why drift is checked *before* training

Training writes a new reference profile from the latest data. If drift were checked *after* training, it would compare the latest data with a profile built from that same data, and report "stable" every single time. It would detect nothing, ever. Checked *before* training, it answers the useful question: **has the world moved since the model that's serving now was fitted?**

The flows

COMMAND	FLOW
<code>python -m src.orchestration.flows run</code>	The full pipeline once, now (<code>make pipeline</code>)
<code>... run --drift</code>	Same, after injecting a behavioural shift, to show the drift path end to end (<code>make pipeline-drift</code>)
<code>... monitor --cutoff 2017-02-28</code>	The drift step alone: check, report, never retrain (<code>make monitor</code>). Used on the real data, where retraining would replace the champion
<code>... backfill</code>	Train once per historical cutoff, oldest first
<code>... serve --cron "0 3 * * *"</code>	Serve the pipeline on a nightly schedule

Prefect gives each step retries, structured logs and a run history. With no Prefect server configured, it starts a temporary local one automatically.

"Needs attention" is not failure

A run is flagged `needs_attention` when a challenger was rejected, drift is significant, or the fairness audit fails. None of these fails the pipeline. A rejected challenger means the gate worked; drift means the world moved. Failing the run would teach whoever is on call to ignore failures. Instead there's an `info` alert and a line in the report.

Idempotency

An operation is **idempotent** if running it twice gives the same result as running it once. For a nightly job this matters: when a run fails halfway and is re-run, it must not double every subscriber's history. The pipeline leaves a populated warehouse alone, and the loaders truncate before writing rather than appending. CI runs the pipeline twice and checks the second run changes nothing and **rejects** the identical model it trains.

Reproducibility: DVC and params.yaml

DVC (Data Version Control) describes the pipeline as stages in `dvc.yaml`, each with its command, inputs, parameters and outputs:

STAGE	DEPENDS ON	PRODUCES
simulate	simulator code; <code>simulation</code> params	<code>src/data/warehouse.db</code>
train	the warehouse; feature, training, cost, calibration, fairness and profile code; features/split/model/threshold/costs/capacity/fairness params	model.joblib, metadata.json, reference_profile.json, metrics.json

`dvc repro` re-runs **only the stages whose inputs changed**, and `dvc.lock` records the exact hash of every input and output. `make metrics-diff` compares the current metrics with `main`. Every tunable number lives in `params.yaml` (windows, split, model hyperparameters, costs, drift thresholds, fairness rule, promotion margin), and any of them can be overridden with an `SDD_*` environment variable. Changing the model is a reviewable change to one file, not an edit buried in code.

The model's own metadata records its training time, parameters, split sizes and **library versions**, and `constraints-model.txt` pins the serving image to those exact versions (Chapter 14).

SAY THIS

"Prefect runs the pipeline as a flow (ingest, drift, train and gate, report) with retries and a nightly schedule. Drift is checked before training, because afterwards it would compare the data with itself. The pipeline is idempotent, and CI proves it by running it twice. DVC and params.yaml make every result reproducible: every setting is in one file, and only changed stages re-run."

CHAPTER 13

CI/CD: testing the system, not just the code

Seven GitHub Actions jobs run on every push. Most CI proves the code compiles and the unit tests pass; this CI proves the system behaves.

The seven jobs

JOB	WHAT IT PROVES
1 • Lint and test	ruff finds no problems; all 434 tests pass; a model trains end to end; the decision-quality report is produced and honest
2 • Train via the warehouse (SQLite)	Simulate a warehouse, train on a temporal split, run the gate, serve the promoted model, smoke-test <code>/predict</code> and the Prometheus endpoint
3 • Train via the warehouse on Postgres	The same on a real Postgres service container, then all eight demo queries run against it
4 • The orchestrated pipeline	First run promotes; second run is idempotent and rejects an identical challenger ; a distinct challenger shadow-scores; injected drift is detected
5 • Streaming scorer	The consume–score–produce loop, including poison messages that must be dead-lettered
6 • Build Docker image	The image builds and serves: <code>/health</code> , the dashboard and <code>/predict</code> all answer
7 • Publish (CD)	On a push to <code>main</code> , once all six pass: build and push the image to the GitHub Container Registry

Continuous delivery

The publish job pushes `ghcr.io/markandeyavarma3-lab/subscriber-dropout-api` with two tags: `latest`, and the **commit hash**. The commit tag means any deployment can be traced back to exactly the code that built it, and rolled back by deploying the previous hash. An image that failed any test can never become `latest`. The image is public; it can be pulled without logging in. It isn't deployed to a cloud cluster automatically (see Chapter 16), so the honest description is **continuous delivery of a tested image**, not continuous deployment to production.

The seventeen days CI was red

Every push from 4 to 23 September failed CI, while every test passed on the development machine, and nobody noticed. The cause: a test asserted that every file a DVC stage depends on exists on disk. `train` depends on `warehouse.db`, which is git-ignored data. It exists on any machine that has built the warehouse, and never on a fresh checkout, which is exactly what CI is. Because the test job gates the others, the warehouse, pipeline, streaming and Docker jobs **hadn't run at all** in that time.

It was found by running the suite on a **fresh clone** with CI's dependency versions, rather than in the working directory. The rule is now "exists in the repo, *or* is produced by a stage", which still catches the typos it was written for, and a new test proves that. Two lasting changes came out of it:

- `make demo-check` now checks the **latest CI run on GitHub** is green.
- The lesson: a test that passes only on your machine is not a passing test.

The test suite

AREA	TESTS	COVERS
Monitoring	38	PSI, drift verdicts, profiles, rolling window
Evaluation	38	Metrics, calibration, costs, fairness, capacity
Streaming	37	Parsing, dead-lettering, commit ordering, producer
API	37	Endpoints, validation, degradation, the dashboard's structure
Prometheus / alerting	36	Metric names, alert rules, routing, dashboard queries scoped and non-overlapping, no credentials
External data	28	KKBox mapping, contract, orphans, streaming large files
Point-in-time	24	Leakage: future events can't change features; same-day signups
Orchestration	22	Pipeline steps, report, needs-attention logic
Training	21	Training, threshold tuning, artifacts, CLI
Registry	20	Logging, aliases, the gate's margin
Shadow, explain, features, DVC, Postgres copy	67	Shadow never affects responses; SHAP additivity; feature building; DVC stage chain; bulk-copy slicing

That's 368 test functions, 434 tests once parametrised cases are counted.

SAY THIS

"Seven jobs on every push. Beyond unit tests, CI trains on SQLite and on Postgres, runs the pipeline twice to prove it's idempotent, checks an identical challenger is rejected, injects drift and checks it's caught, and sends poison messages to the stream scorer. On main it publishes the image, tagged with the commit. And honestly: CI was once red for 17 days without anyone noticing, so now the demo checklist checks it."

CHAPTER 14

Deployment: Docker, Compose and Kubernetes

Eight services, one command, identical on every machine, and manifests for the step after that.

The Docker image

The `Dockerfile` is **multi-stage**. A builder stage installs dependencies and trains a model; a slim runtime stage copies only what's needed to serve. Smaller image, smaller attack surface. The service runs as a **non-root user** and has a built-in health check.

The image trains its own model at build time on *synthetic* data, because the real model file is git-ignored and a build must work from a clean checkout. For the real stack, the promoted KKBox model is **mounted over it, read-only**, and the service is told to load it (`SDD_MODEL_SOURCE=local`). Before this was fixed, `docker compose up` silently served the synthetic model, so the dashboard and Grafana showed a different model from the one the registry promoted.

Pinning the model's libraries

A pickled scikit-learn model should be loaded by exactly the scikit-learn that trained it.

`requirements.txt` gives version *ranges*, so each rebuild pulled whatever was newest; one rebuild quietly moved scikit-learn from 1.9.0 to 1.9.1. `constraints-model.txt` now pins scikit-learn, pandas and NumPy to the training versions, and the demo checklist compares the running container's versions with the model's metadata.

Docker Compose: the full stack

SERVICE	IMAGE	PORT	ROLE
postgres	postgres:16-alpine	5432	The event warehouse (and MLflow's own database)
subscriber-api	built from the Dockerfile	8000	API and dashboard
stream-scorer	same image	8001	Kafka consumer; metrics endpoint
redpanda	redpanda v24.2.7	19092	Kafka-compatible broker
prometheus	prom/prometheus v2.54.1	9090	Metrics and alert rules
alertmanager	prom/alertmanager v0.27.0	9093	Alert routing
grafana	grafana 11.2.0	3000	Dashboard (anonymous read-only viewing)
mlflow	mlflow v3.1.1	5001	Tracking server for the containerised stack

Images are pinned to exact versions, so the stack is the same next year. Data lives in named volumes, which survive restarts. For the demo, the MLflow UI is served from the real training history in `mlflow.db` on port **5050**; port 5000 is taken by macOS's AirPlay Receiver.

Kubernetes

`deploy/kubernetes/` holds the step after Compose:

- **Deployments** for the API and the stream scorer, each with 2 replicas, CPU and memory requests and limits, and a non-root security context.
- **Probes:** liveness on `/health`, readiness on `/ready`, and a startup probe so a slow model load isn't mistaken for a crash.
- A **HorizontalPodAutoscaler**: 2 to 10 API replicas, targeting 70% CPU.
- A **PodDisruptionBudget**: at least one API pod stays up during node maintenance.
- A **ConfigMap** for settings, so the same image runs everywhere and only configuration changes.

These manifests are **provided, not deployed**: there's no free cluster large enough for the full stack. That's stated openly, and deploying to a managed cluster is listed as future work.

SAY THIS

"Everything runs in Docker: eight services from one Compose file, images pinned to exact versions. The serving image runs as a non-root user, and the promoted model is mounted into it read-only, with its libraries pinned to the training versions. For the next step there are Kubernetes manifests with health probes, an autoscaler and a disruption budget; they're written but not deployed, because no free cluster fits the stack."

CHAPTER 15

Running the demo

Everything needed to bring the system up on the Mac, prove it's healthy, and show it.

The four commands

COMMAND	WHEN	WHAT IT DOES
<code>make demo-prepare</code>	Once, the day before	Builds the Docker image and writes the drift report, so nothing slow happens on the day
<code>make demo</code>	Before presenting	Starts Docker if needed, starts the services, waits until each one is <i>really</i> serving, starts the MLflow UI, sends warm-up traffic, prints every URL
<code>make demo-check</code>	Straight after	The pre-demo checklist: 19 checks, green or red
<code>make demo-down</code>	Afterwards	Stops everything and keeps all data (never deletes volumes)

`make demo` sends warm-up traffic (300 predictions through the batch endpoint and 60 events onto the Kafka topic) because a freshly started Prometheus and Grafana show "No data" everywhere, which looks broken to anyone who doesn't know why.

The 19 checks

CHECK	WHY IT MATTERS
6 × each web UI responds	Dashboard, API docs, MLflow, Grafana, Prometheus, Alertmanager
Serving the promoted KKBBox model	The served model's training date matches the real artifact, not the synthetic fallback
Container libraries match training	scikit-learn, pandas and NumPy in the container equal the model's recorded versions
3 × preset scores	At-risk 0.6037 high, healthy 0.0005 low, borderline 0.0501 medium. If the model changed, these move
5 × Postgres row counts	Every warehouse table holds exactly the rows it should
Drift report present	Grafana's drift panels have real data
Prometheus scraping every target	API, stream scorer, Alertmanager and Prometheus itself all up

CHECK	WHY IT MATTERS
Latest CI run on main is green	Because the demo sends the examiner to GitHub Actions

Dashboard	<code>http://127.0.0.1:8000</code>
API docs	<code>http://127.0.0.1:8000/docs</code>
MLflow	<code>http://127.0.0.1:5050</code>
Grafana	<code>http://127.0.0.1:3000</code> (viewing needs no login)
Prometheus	<code>http://127.0.0.1:9090</code> (alerts at <code>/alerts</code>)
Alertmanager	<code>http://127.0.0.1:9093</code>
Postgres	<code>localhost:5432</code> , database <code>warehouse</code> , user and password <code>subscriber</code>

Using the database, step by step

TablePlus is a window onto the Postgres database. It doesn't store any data itself: the 82.8 million rows live in Postgres, inside Docker. TablePlus shows them, and lets you run SQL against them. It's free (the trial never expires; it limits you to 2 tabs and 2 windows), and the copy on this Mac is signed by TablePlus Inc and notarized by Apple.

1 • Connect (once)

1. Make sure the stack is running: `make demo` , or check Docker Desktop shows `postgres` running.
2. Open TablePlus, click **Create Connection**, choose **PostgreSQL**, then **Create**.
3. Fill in, leaving everything else as it is: Name `Subscriber Warehouse` , Host `127.0.0.1` , Port `5432` , User `subscriber` , Password `subscriber` , Database `warehouse` .
4. Click **Test** (the fields turn green), then **Connect**. Next time, double-click the saved connection.

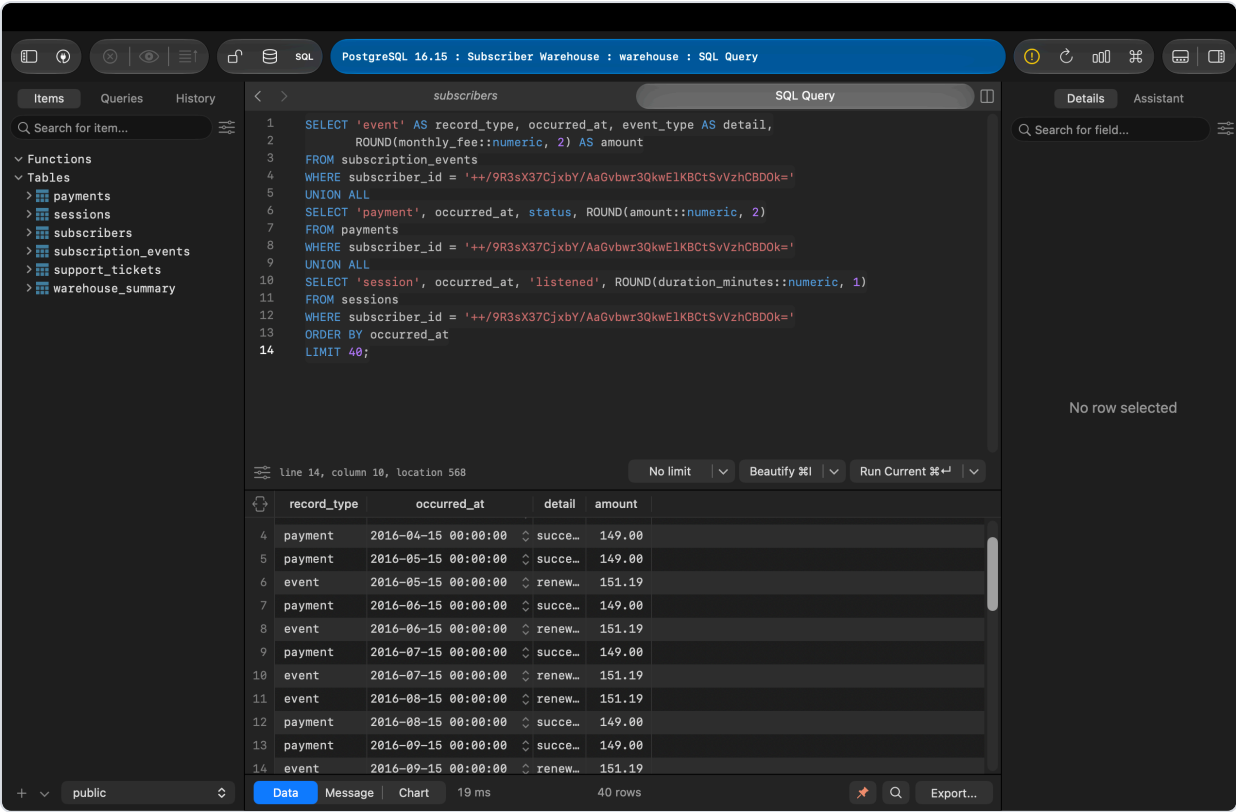


Figure 15.1: TablePlus connected to the warehouse, running query 4: one subscriber's full history, 40 rows in 19 ms.

2 · What you're looking at

PART OF THE SCREEN	WHAT IT IS
Blue bar at the top	Where you are: PostgreSQL 16.15 : Subscriber Warehouse : warehouse .Postgres version, connection name, database
Left sidebar → Tables	The six tables. Click one to open it
Tabs above the editor	A table tab (e.g. subscribers) and a SQL Query tab. The free version allows two
Editor (middle)	Where you type or paste SQL
Run Current ⌘↵	Runs only the query your cursor is in. The one to use in a demo
Results grid (bottom)	The rows the query returned. Scroll or drag column edges to widen
Bottom bar	Data (the grid), Message (errors), Chart; the query time (19 ms); the row count (40 rows); Export
Right panel · Details	Click any row to see all its fields listed vertically: handy for long values

3 · Browse a table

Click a table in the sidebar to see its rows. Switch the table view to **Structure** (at the bottom of the table tab) to see its columns, their types and its indexes, the proof the schema is what the book describes. Use the filter bar above the rows to narrow them, for example `event_type = cancellation`.

THE ONE THING THAT CAN FREEZE TABLEPLUS

The query editor has a row-limit dropdown ("No limit" in the screenshot). **Never run `SELECT * FROM sessions without a LIMIT`**: that asks TablePlus to fetch and draw 38 million rows. Always end a query on a big table with `LIMIT 20`, or set the dropdown to a limit before browsing.

4 · Run the demo queries

1. Press `⌘E` (or click the SQL Query tab).
2. Open `DEMO_QUERIES_POSTGRES.sql` in VS Code and copy one numbered block.
3. Paste it in TablePlus, click anywhere inside it, press `⌘↵`.
4. For the next query: `⌘A` to select everything in the editor, paste over it, `⌘↵`.

#	QUERY	WHAT IT SHOWS
1	Warehouse summary	82.8 million rows across the tables
2	Subscribers	44-character anonymised IDs
3	The event log	Timestamped lifecycle events
4	One subscriber's history	Events, payments and sessions for one person, from three tables, in milliseconds
5	The cleaning log	The cleaning decisions and why
6	Plan mix	Events, average fee and auto-renew by plan
7	Lifecycle breakdown	Signups, renewals, cancellations
8	Acquisition channels	The opaque <code>via_n</code> codes, carried through honestly
9	Table sizes	16 GB on disk; <code>sessions</code> alone is 7.3 GB
10	Indexes	The <code>(subscriber_id, occurred_at)</code> index behind the fast lookups
11	EXPLAIN ANALYZE	Proof the index is used: 63 sessions found among 38 million in about 0.2 ms

#	QUERY	WHAT IT SHOWS
12	Cancellations per month	Cancellations each month: 19,477 in June 2016 up to 26,967 in February 2017. A count, not a rate, so it grows with the subscriber base too

Queries 1–8 are the planned demo; 9–12 are ready for when an examiner asks something unplanned, such as "how big is it?", "why is it fast?", or "is churn changing over time?". The Postgres versions differ from the SQLite ones in two places, because Postgres is stricter: `ROUND` needs a `numeric` value, and booleans are counted with `FILTER` rather than summed as integers.

5 · Safe habits

- **Only run `SELECT` queries in a demo.** Never `DELETE`, `UPDATE`, `DROP` or `TRUNCATE`.
- TablePlus lets you edit a cell by double-clicking it. Edits aren't saved until ⌘S, so if you change something by accident, press ⌘Z or close without saving.
- If a query seems stuck, click the stop button in the toolbar rather than quitting.
- Ignore *Purchase* and *Activate*; the free version is enough.

If something breaks

SYMPTOM	DO THIS
A check fails	Read its detail column. Most failures are a service still starting: wait 30 seconds and run it again
Docker won't start	Open Docker Desktop by hand, wait for it to settle, run <code>make demo</code> again
TablePlus "connection refused"	<code>docker compose up -d postgres</code> , wait 10 seconds, reconnect
Grafana says "No data"	Traffic ages out. Click a few presets on the dashboard, or run <code>make demo</code> again
CI check says "rate limit"	GitHub allows 60 unauthenticated requests an hour; open the Actions page instead
Anything else, live	Don't debug in front of the examiner. Switch to the screenshots in <code>docs/viva/screenshots/</code>

CHAPTER 16

Results, limitations and future work

What the system achieves, where it falls short, and what would come next, stated plainly.

What was achieved

- A complete MLOps system on **real** data: 31.3 GB raw, 82.8 million warehouse rows.
- A leak-free training pipeline, verified by tests and by cross-checking two databases row for row.
- A model about **4.8× better than random** at finding churners (PR-AUC 0.073 against a 0.015 base), and **well calibrated** (ECE 0.004).
- A registry with a promotion gate, proven in CI to reject an identical challenger.
- Explained predictions, a dashboard, and streaming inference with a dead-letter queue.
- Monitoring with PSI drift, 15 alert rules and a Grafana dashboard that reports only true things.
- Seven CI/CD jobs, 434 tests, and a published, commit-tagged image.
- The whole stack started and verified with two commands.

Limitations

LIMITATION	WHY, AND WHAT IT MEANS
The model is modest	PR-AUC 0.073, recall 16.8%. KKBox lacks strong signals: two inputs are always zero, and the strict churn definition gives few positives. It's far better than random, but it wouldn't catch most churners.
The fairness audit fails	Near-random on the standard plan (949 subscribers). Reported and alerted, not fixed.
Cost figures are placeholders	The cost-optimal threshold (0.43) is illustrative; the served threshold (0.05) is the F1 choice.
Stricter churn label than Kaggle's	Explicit cancellations only, not quiet lapses. Results aren't comparable with the competition's leaderboard.
Calibration is reported, not applied	Isotonic calibration would shave 0.003 off an already-small error; the pipeline tries it on every run but doesn't switch it on.
Not deployed to a public cloud	A 15 GB warehouse plus MLflow, Kafka and Grafana fits no free tier. It runs on Docker locally; Kubernetes manifests are written but not deployed.
Continuous delivery, not deployment	Every merge publishes a tested image; nothing deploys it to production automatically.
No real challenger yet	Shadow scoring is built and tested in CI, but idle in the demo.

Future work

1. **Richer features** from the full 392-million-row listening log: trends (listening falling week on week), skip rates, completion rates.
2. **Real offer costs**, so the threshold is chosen by money rather than F1.
3. **The competition's churn definition**, trained alongside this one, to compare.
4. **Address the fairness failure**: more standard-plan data, a per-plan threshold, or a model with plan-specific interactions.
5. **Deploy to a managed Kubernetes cluster**, with CI deploying each published image to a staging environment.
6. **A real challenger** (for example, a model with the new features) running in shadow before any promotion.
7. **A feature store**, so training and serving read features from one place.

SAY THIS

"The engineering is complete and tested, and the model is modest: about 5× better than random, because the data lacks strong signals. The fairness audit fails for one plan, the costs are placeholders, and it isn't on a public cloud. I'd add listening-trend features from the full logs, real offer costs, and a managed Kubernetes deployment next."

CHAPTER 17

What building it taught

The bugs and surprises, collected in one place. "Tell us about a problem you found" is a classic viva question; each of these is a real, specific answer.

PROBLEM	HOW IT WAS FOUND	LESSON
ROC-AUC rose while PR-AUC fell on real data	Moving from synthetic to real data	Choose metrics for the class balance you'll actually face
IDs declared 32 characters; real ones are 44	Loading real data; Postgres would reject every row	SQLite forgives what Postgres won't; test on the real database
Same-day signups counted as "before" the cutoff	Comparing SQLite and Postgres snapshots row for row	Text-stored dates compare as text; cross-check two implementations
A subscriber cap that didn't cap anything	A single cutoff never finished on real data	A <code>LIMIT</code> on one step doesn't bound the others; push the filter into every aggregate
An explanation that contradicted its input ("auto-renew enabled" when it was off)	Reading real explanations	For booleans, state the value, not the direction of the effect
Preset examples silently inverted (at-risk scored low)	Looking at the dashboard after retraining	Tests that only compare two outputs can pass while both are wrong
The container served a synthetic model	Checking <code>/model-info</code> in the running stack	Verify the deployed artifact, not the code
False alerts, "REJECTED" and overlapping panels in Grafana	A screenshot, with every check green	A passing check isn't a correct screen; look at it
scikit-learn drifted from 1.9.0 to 1.9.1 on rebuild	A new checklist item comparing versions	Pin what the model depends on
CI red for 17 days	Running the suite on a fresh clone	A test that passes only on your machine isn't passing

THE COMMON THREAD

Almost every one of these was invisible to the automated tests at the time and found by **checking the real thing**: the real data, the real database, the running container, the actual screen, a clean checkout. The lasting fixes turned each discovery into a test or a checklist item, so it can't silently come back.

PART II

Viva question bank

Questions an examiner is likely to ask, grouped by topic, with short answers. **CORE** marks the most likely; **TRICKY** marks the ones that probe a weakness or the edge of your understanding.

SECTION A

The project and its purpose

Q1 Explain your project in one minute. CORE

It predicts which subscribers of a music-streaming service will cancel in the next 30 days, using real KKBox data: 31 GB, 6.8 million subscribers. The model is the small part. The project is an MLOps system around it: a Postgres event warehouse that can't leak the future, a registry where a model only goes live by beating the current one on PR-AUC, an API and a Kafka stream scorer that explain every prediction, drift monitoring and alerting, a scheduled retraining pipeline, and CI/CD that tests the whole system on every push.

Q2 Why did you choose churn prediction? CORE

It's a real business problem with a clear decision attached (who gets a retention offer), so it exercises everything MLOps is meant to handle: a rare event, costs that differ by type of mistake, a model that goes stale as behaviour changes, and a need to explain decisions to the people acting on them. There's also a public, realistic dataset for it.

Q3 What is MLOps, and how is it different from DevOps? CORE

MLOps applies DevOps practices (automation, testing, monitoring, reproducibility) to machine-learning systems. The difference is that an ML system depends on **data** and a **trained model** as well as code. So on top of versioning and testing code, MLOps has to version data and models, validate data, gate which model goes live, and monitor for the model quietly getting worse as the data changes. That last one is a failure mode ordinary software doesn't have.

Q4 What are the main components of your system? CORE

Four stages plus automation. **Data:** KKBox ingestion and a Postgres event warehouse. **Model:** point-in-time features, gradient boosting, a decision-quality audit, and an MLflow registry with a promotion gate. **Serving:** a FastAPI service with SHAP explanations and a dashboard, plus a Kafka stream scorer. **Monitoring:** Prometheus, Alertmanager, Grafana and PSI drift detection. **Automation:** Prefect for the pipeline, DVC for reproducibility, GitHub Actions for CI/CD.

Q5 Who would use this system, and how?

A retention team. Each day they'd get the subscribers flagged as at risk, with the reasons, and decide who gets an offer. Engineers would use the monitoring to know whether the model can still be trusted, and the pipeline would retrain it on a schedule.

Q6 What makes this more than a Jupyter-notebook project? CORE

A notebook trains a model once. This system runs continuously and safely: data is validated before loading; training is reproducible and can't leak the future; a new model can't go live unless it's measurably better; predictions are served with explanations and graceful failure; the model is monitored for drift and alerts fire; and 434 tests plus seven CI jobs check the whole system on every change.

Q7 What is your single most important design decision? CORE

Storing data as timestamped events and building features point-in-time. It's what makes every metric trustworthy: the model is trained only on information it would really have had at prediction time. Without it, every number in the project could be inflated by leakage.

Q8 What was the hardest part?

Working with real data at scale. The 392-million-row listening log needed a bounded window; the orphan transactions needed a defensible decision; and a feature query that ran fine on synthetic data never finished on the real data, because a subscriber cap on one step didn't bound the others. The second hardest was making the monitoring report only true things, which I found by looking at the dashboards rather than trusting green tests.

Q9 What did you learn? CORE

That passing tests aren't the same as a working system. Nearly every serious bug was invisible to the tests at the time and found by checking the real thing: the real database, the running container, the actual screen, a fresh clone. Each discovery then became a test or a checklist item. And that metric choice matters enormously on imbalanced data.

Q10 If you had one more month, what would you do?

Add listening-trend features from the full 392M-row log (weekly decline, skip rates), which is where the missing signal most likely is; replace the placeholder costs with real ones so the threshold is set by money; and deploy to a managed Kubernetes cluster with CI deploying each image to staging.

SECTION B

Data and the warehouse

Q11 What dataset did you use, and where is it from? CORE

The KKBox churn dataset from the WSDM Cup 2018 competition on Kaggle. KKBox is a Taiwanese music-streaming service. Three files, 31.3 GB: members (6.77M subscribers), transactions (21.5M payments and plan changes) and daily listening logs (392M rows).

Q12 Is the data real or synthetic? CORE

The trained model and the warehouse use **real** KKBox data. There's also a synthetic data generator, used for fast tests and CI, because CI can't download 31 GB. Every pipeline works identically on both; the switch is only which data is loaded.

Q13 Is there any personal data? What about privacy?

No directly identifying data. KKBox anonymised the export: subscribers are 44-character hashes, with no names, emails or phone numbers. The members file has city, age and gender, which aren't used as model inputs. It's still behavioural data about people, so in a real deployment it would need a lawful basis, access control and a retention policy.

Q14 Why store events rather than one row per subscriber? CORE

Because training needs the past. To build a training example "as of 1 November 2016", I need each subscriber exactly as they were that day. Events with timestamps can answer that for any date by filtering on time; a table of running totals can't, because the totals already include the future.

Q15 Describe the warehouse schema. CORE

Five tables: `subscribers` (one per person: signup date, channel), `subscription_events` (signup, renewal and cancellation, with plan, fee and auto-renew), `payments`, `sessions` (one per listening day) and `support_tickets` (empty for KKBox). Each event table has an index on (`subscriber_id`, `occurred_at`). 82.8 million rows in total.

Q16 Why that index?

Almost every query asks "this subscriber's events before this date". A composite index on (`subscriber_id`, `occurred_at`) answers exactly that: find the subscriber, then read their events in time order. One subscriber's full history comes back in 17 milliseconds out of 82.8 million rows.

Q17 What cleaning did you do? CORE

Dropped 2.66 million orphan transactions; widened the ID column from 32 to 64 characters; clipped negative listening durations to zero; normalised prices to a monthly rate; mapped plan lengths to tiers; and bounded the listening history to October 2016 – February 2017.

Q18 Why did you drop the orphan rows instead of fixing them? CORE TRICKY

432,623 subscribers appear in transactions but not in the members file. The only way to "fix" them is to invent a signup date, and the earliest possible one is their first transaction, which lands on the first day of the log. So they almost certainly signed up *before* the data begins. Inventing a date would make loyal long-term customers look brand new, and the model treats "new" as risky. That would silently corrupt 432,623 subscribers' tenure. Dropping them is visible and countable.

If they push: "Isn't 18% a lot to drop?" Yes, and it's reported, not hidden. The alternative corrupts a feature for all of them, which is worse than losing them. The remaining 6.77M subscribers are still plenty.

Q19 Why only load sessions from October 2016?

Features look at the 30 days before a cutoff, and the cutoffs run from November 2016 to February 2017. Loading all 392 million rows would take over 7 hours for data no feature uses. Bounding it to 38.2 million rows keeps every needed day.

Q20 KKBox has no plan tiers. Where do basic/standard/premium come from? TRICKY

From plan length: up to 31 days is basic, 32–180 standard, 181+ premium. It's an assumption, and it's documented as one. KKBox only has durations and prices, and longer commitments are the closest analogue to a higher tier.

Q21 How do you validate incoming data?

With a data contract (`contract.py`): required columns, parseable types, value ranges, non-empty IDs, and events that refer to known subscribers. A dry-run mode validates everything and writes nothing, so a problem in a 30 GB file shows up in minutes, not hours into a failed load.

Q22 Why Postgres? Why not just SQLite? CORE

Postgres is a real database server: strict types, parallel queries and a much better query planner. The same feature query took 506 seconds on SQLite and 6.8 seconds on Postgres, about 74× faster. It also caught bugs SQLite silently accepted. SQLite stays as a zero-setup fallback for development and tests.

Q23 You didn't install Postgres. Where is it running? CORE

In a Docker container, from the official `postgres:16-alpine` image, defined in `docker-compose.yml`. Its port 5432 is published to the Mac, so TablePlus and the pipeline connect to `localhost:5432` as if it were installed normally. The data lives in a Docker volume, so it survives restarts. That's how companies run databases too: the same version everywhere, nothing to install.

Q24 How did you get 82.8 million rows into Postgres?

With Postgres's bulk `COPY`. SQLite exports each table as CSV, which is piped straight into `COPY` inside the container in 5-million-row slices, with no temporary files. Indexes are dropped during the load and rebuilt once at the end. It took about 7.5 minutes, and every table's row count was verified equal to the source.

Q25 Why not insert the rows with normal INSERT statements?

82.8 million individual inserts through Python would take hours: every row is a separate parameter set and index update. `COPY` is Postgres's bulk-load path; it parses the stream directly and runs at hundreds of thousands of rows per second.

Q26 How does the same code run on two different databases?

Through SQLAlchemy. The schema is defined once in Python, and the queries use bound parameters and standard SQL. Date arithmetic is done in pandas rather than SQL, because the two databases write date maths differently. One environment variable, `SDD_DATABASE_URL`, chooses the database.

Q27 What is a data warehouse, and is this one?

A data warehouse is a database organised for analysis: historical, integrated data from source systems, optimised for reading. This is a small one: an **event store** of integrated KKBox history, designed so any past state can be reconstructed for analysis and training. It isn't a column-store analytics engine like BigQuery or Snowflake, which would be the choice at larger scale.

Q28 Two of your nine inputs are always zero. Why keep them? TRICKY

KKBox has no customer-service data, so support tickets are always 0, and it only records a transaction when money moved, so payment failures are always 0. The model can't learn from a constant. They remain because the API's input format requires them, and the schema is designed for a business that does have that data. The dashboard says so openly and never offers them as levers.

SECTION C

Features and data leakage

Q29 What is data leakage? CORE

Training data that contains information the model wouldn't have when it's really used, typically information from the future. The model then looks excellent in testing, because it's partly reading the answer, and fails in production. It's the most common way ML projects overstate their results.

Q30 How did you prevent leakage? CORE

Point-in-time features. For a cutoff date T, every feature uses only events strictly before T (the previous 30 days, or 180 for payments); the label only looks at the 30 days from T. The windows never overlap. A test adds future events and checks no feature changes. And the data is split by time, never randomly.

Q31 Give an example of leakage that would have been easy to introduce. CORE

Computing "sessions in the last 30 days" as of *today* for everyone, then using labels from last month. For a subscriber who churned last month, that feature covers the period after they left: zero sessions, which perfectly predicts the label. The model would look superb and be useless.

Q32 Define your label exactly. CORE

1 if the subscriber has a cancellation event (KKBox's `is_cancel`) in the 30 days from the cutoff, otherwise 0. Subscribers who had already cancelled before the cutoff are excluded, because they can't churn again.

Q33 Kaggle defines churn differently. Why didn't you use their definition? TRICKY

The competition counts anyone who doesn't take a new subscription within 30 days of expiry, which includes quiet lapses. Mine counts explicit cancellations. It's a stricter, event-based definition that fits the warehouse model, but it gives fewer positives (about 1.5%) and isn't comparable with the leaderboard. Training on the competition's definition too is in the future work.

Q34 What features does the model use? CORE

Nine inputs: tenure, plan type, monthly fee, listening days in the last 30 days, days since last activity, support tickets, payment failures, discounts used, and auto-renew. Derived from them: tenure in months, recency ratio (days inactive ÷ tenure), engagement-recency score, fee per session, discount dependency, a friction score, per-month rates, and a dormant flag. Plan type is one-hot encoded.

Q35 Why is "last active" computed over all history, not just 30 days?

A subscriber absent for 90 days has no sessions in a 30-day window at all. If "last active" only looked at the window, that would read as "no data" rather than "long absent", erasing one of the strongest churn signals. So it looks back across all history before the cutoff, and is capped at tenure.

Q36 Why are dates handled in pandas instead of SQL?

SQLite and Postgres write date arithmetic differently. Passing dates as parameters and doing the subtraction in pandas keeps a single query that runs identically on both.

Q37 Tell me about the bug you found comparing SQLite and Postgres. CORE

The same training snapshot differed by one subscriber out of 1,104. SQLite stores dates as text, and the query compared `'2016-12-01' < '2016-12-01 00:00:00'`. As strings that's true, because a shorter prefix sorts first. So subscribers who signed up *on* the cutoff day were counted as before it. No future data reached a feature, but they took slots in a capped sample. I wrote a failing test first, fixed it by comparing against a real date, and the two databases then matched exactly.

Q38 Why split by time rather than randomly? CORE

A random split mixes later rows into training and earlier rows into testing. The model learns from the future and is graded on the past, so the score measures memory, not prediction. Splitting by time mirrors real use: train on what has happened, judge on what happens next. Train: Nov–Dec 2016; validation: 31 Dec 2016; test: 29 Jan 2017.

Q39 Why not use k-fold cross-validation? TRICKY

Standard k-fold shuffles rows, which breaks time order and leaks the future, as above. For time-dependent data the right form is **time-series (forward-chaining) validation**: train on earlier periods, validate on later ones. The fixed train/validation/test split by date is the simplest version of that.

Q40 Do training and serving compute features the same way? CORE

Yes. The derived features and preprocessing are one scikit-learn pipeline, saved with the model. Serving passes the raw nine inputs through the same pipeline, so training–serving skew from two different implementations can't happen.

Q41 What is training-serving skew?

When the features a model sees in production are computed differently from those it was trained on, for example a different formula, unit or default. The model then silently performs worse live than in testing. Sharing one pipeline object between training and serving prevents it.

Q42 What preprocessing is applied?

A scikit-learn ColumnTransformer: missing numeric values are imputed and scaled, and the plan type is one-hot encoded. Scaling doesn't matter for tree models, since trees split on thresholds and scaling doesn't change their order. It's harmless, and it keeps the pipeline usable with other model types.

SECTION D

The model

Q43 Which algorithm did you use, and why? CORE

Scikit-learn gradient boosting. It's among the strongest methods for tabular data, handles mixed numeric and categorical inputs, captures non-linear effects without hand-built features, and supports SHAP's exact tree explainer, so every prediction can be explained exactly.

Q44 How does gradient boosting work? CORE

It builds decision trees one after another. Each new tree is fitted to the *errors* of the ensemble so far (technically, the gradient of the loss), and its predictions are added in, scaled down by the learning rate. Many small corrections add up to a strong model. For classification it works in log-odds and converts to a probability at the end.

Q45 Gradient boosting vs random forest? CORE

A random forest builds many deep trees *independently*, on random samples, and averages them, which mainly reduces variance. Gradient boosting builds shallow trees *sequentially*, each correcting the last, which mainly reduces bias. Boosting is usually more accurate on tabular data but more sensitive to its settings and easier to overfit.

Q46 Why not XGBoost or LightGBM?

They're faster, more tuned implementations of the same idea, and worth trying. At 67,000 training rows, scikit-learn's version trains in seconds, keeps dependencies simple, and works with SHAP's exact explainer. The model is swappable because the registry and gate don't care which algorithm produced a model.

Q47 Why not deep learning? TRICKY

On structured, tabular data with a handful of features, tree ensembles consistently match or beat neural networks, while being faster, needing less data, and being easier to explain. Deep learning shines on images, text and sequences. A sequence model over the raw daily listening log could be interesting future work.

Q48 Explain your hyperparameters.

300 trees at learning rate 0.05: many small steps generalise better than few large ones. Max depth 3: shallow trees capture interactions without memorising. Subsample 0.9: each tree sees 90% of rows, which adds randomness and reduces overfitting. Min samples per leaf 20: no decision rests on a handful of subscribers.

Q49 Did you tune the hyperparameters? **TRICKY**

No systematic search: they're sensible, conservative defaults kept in `params.yaml`. The limiting factor here is the features, not the settings. A search (on the validation set, never the test set) would be a fair next step, and the registry's gate would decide whether the tuned model earns promotion.

Q50 How do you know the model isn't overfitting?

It's judged on a later, unseen time period. Validation PR-AUC (0.052) and test PR-AUC (0.073) are in the same range, and ROC-AUC is 0.834 against 0.835. An overfitted model would do much better on training data than on these. The shallow trees, subsampling and leaf minimum also limit memorisation.

Q51 What is overfitting? Underfitting? **CORE**

Overfitting: the model learns noise specific to the training data, so it scores well there and poorly on new data. Underfitting: it's too simple to capture the real pattern and scores poorly everywhere. The bias–variance trade-off is the balance between the two.

Q52 How did you handle the class imbalance? **CORE** **TRICKY**

Not with resampling or class weights. Through the **metric** (PR-AUC, which is sensitive to the rare class) and the **threshold** (0.05 instead of 0.5). That was deliberate: oversampling or reweighting distorts the predicted probabilities, and this project uses the probabilities directly for costs and calibration. Keeping them honest matters more than a nicer-looking recall.

If they push: "Wouldn't SMOTE help?" It would change the operating point, not the information available: it creates synthetic positives from existing ones. It could be tried as a challenger, and the gate would judge it on PR-AUC, but it would need recalibrating afterwards.

Q53 What is the most important feature, and does it make sense? **TRICKY**

Auto-renew, at 28% of importance, and surprisingly it *raises* risk. On KKBox, auto-renew goes with the cheap, short plans that churn most. It's a real association in the data, not a cause. That's exactly why explanations matter: a human should know when a model's reasoning is a correlation, not something to act on directly.

Q54 Is the model deterministic?

Yes. A fixed random seed (42) controls subsampling, and the split is by date, so retraining on the same data gives the same model. That's also what lets CI check that an identical challenger is rejected.

SECTION E

Metrics and evaluation

Q55 Why not use accuracy? CORE

Only 1.5% of subscribers churn, so predicting "nobody churns" is 98.5% accurate and useless. Accuracy rewards ignoring the rare class. My model's accuracy is 95.6%, *lower* than that useless baseline, because it deliberately flags people.

Q56 Define precision and recall. CORE

Precision: of those flagged, the fraction who really churn ($TP \div (TP + FP)$). **Recall:** of those who really churn, the fraction flagged ($TP \div (TP + FN)$). At the 0.05 threshold: precision 7.6% (85 of 1,125 flagged), recall 16.8% (85 of 506 churners).

Q57 What is F1?

The harmonic mean of precision and recall: $2PR \div (P + R)$. It's high only when both are high, and treats a missed churner and a false alarm as equally bad. Here it's 0.104 at 0.05. That equal treatment is its weakness when the two mistakes cost different amounts, which is why the cost audit exists.

Q58 What is ROC-AUC? CORE

The area under the curve of true-positive rate against false-positive rate across all thresholds. It equals the probability that the model scores a random churner above a random non-churner. 0.5 is random; 1.0 is perfect. This model: 0.835.

Q59 What is PR-AUC, and why is it your main metric? CORE

The area under the precision–recall curve across all thresholds. A random model's PR-AUC equals the positive rate (here 0.015). It's built from precision, which is dominated by the rare class, so it reflects how useful the flags really are when churners are rare. ROC-AUC uses the false-positive *rate*, which divides by the huge number of non-churners and so hides how many false alarms you'd send. Mine is 0.073: about 4.8× random.

Q60 Tell me about ROC-AUC going up while PR-AUC went down. CORE

Moving from synthetic data (about 20% churn) to real KKBox (1.5%), ROC-AUC rose from 0.674 to 0.835 while PR-AUC fell from 0.345 to 0.073. ROC-AUC said "better"; PR-AUC said "much harder", which is the truth for the business, because with rarer churners far more flags are false alarms. A gate on ROC-AUC would have claimed an improvement at exactly the wrong moment. That's why the gate uses PR-AUC.

Q61 PR-AUC 0.073 sounds terrible. Is your model any good? CORE TRICKY

Compare it with the baseline, not with 1.0. Random gets 0.015, so the model is about 4.8× better; its flags are 5× richer in real churners than random picks. But honestly, it's modest: it catches 1 churner in 6. The data lacks strong signals (two inputs are always zero), and my stricter churn definition leaves few positives. The project's contribution is mainly the engineering around the model, and the model can be swapped through the gate.

Q62 Show me the confusion matrix.

At threshold 0.05, on 33,042 test subscribers: 85 true positives, 1,040 false positives, 421 false negatives, 31,496 true negatives. 1,125 flagged, of whom 85 churned; 506 churned in total.

Q63 Why do validation and test scores differ (PR-AUC 0.052 vs 0.073)? TRICKY

They're different months with different churn rates (1.43% vs 1.53%) and only about 500 positives each, so PR-AUC is noisy between periods. ROC-AUC is almost identical (0.834 vs 0.835). The variation is a reason for the promotion margin: small differences between runs can be noise.

Q64 What is a baseline, and what's yours?

The simplest reference a model must beat to be worth anything. Here: random guessing (PR-AUC 0.015) and "predict nobody churns" (98.5% accuracy, zero recall). The model beats random by 4.8× on PR-AUC. A good next baseline would be a simple rule, like "inactive for 30+ days", to show how much the model adds over common sense.

Q65 What is the Brier score?

The mean squared difference between predicted probability and outcome (0 or 1); lower is better. It measures accuracy and calibration of the probabilities together. Here it's 0.0146.

SECTION F

Thresholds, costs, calibration and fairness

Q66 What is the decision threshold, and why 0.05? CORE

The probability above which a subscriber is flagged. 0.5 would flag almost nobody at a 1.5% churn rate. 0.05 is the value that maximised F1 on the validation set, searched from 0.05 to 0.95. It sits at the bottom of that range, so F1 would have liked it even lower.

Q67 Is 0.05 the best threshold? TRICKY

Not by cost. The cost audit prices each mistake and finds the cheapest threshold is 0.43, about £16.5k cheaper on the test set, because at 1.5% churn and 30% offer efficacy almost nobody is worth an offer. But those costs are placeholders. The audit reports the gap rather than silently changing the threshold, because the right choice depends on real business costs.

Q68 Explain your cost model. CORE

A missed churning costs £240 (about a year of revenue). An offer costs £20 and saves 30% of real churners. So flagging a churning costs $£20 + 70\% \times £240 = £188$ (the offer, plus the loss when it fails), flagging a non-churning costs £20, and missing a churning costs £240. Flagging is worth it when $p \times (240 - 188) > (1 - p) \times 20$, which gives $p > 0.278$. All of these are in `params.yaml` and all are assumptions.

Q69 Why does the cost-optimal policy send only 8 offers? TRICKY

Because an offer only helps if the subscriber would really churn *and* the offer works (30%). With a 1.5% base rate, very few subscribers have a churn probability high enough that the expected saving beats a £20 offer. The honest conclusion under these assumptions is that retention offers are barely worth it for this population, not that the model is wrong.

Q70 What does "calibrated" mean? Is your model calibrated? CORE

A model is calibrated if its probabilities mean what they say: among subscribers given 5%, about 5% churn. It matters because the probability feeds cost decisions. Expected calibration error is 0.004, which is very good. It slightly under-predicts on average (1.16% predicted vs 1.53% actual).

Q71 What is expected calibration error?

Predictions are grouped into bins by probability. In each bin, compare the average predicted probability with the actual churn rate, and take the weighted average of those gaps. 0 means perfectly calibrated.

Q72 Did you calibrate the model? **TRICKY**

Every run tries isotonic calibration, fitted on the validation set, and reports whether it would help. It would improve the error by a further 0.003, from an already-small 0.004. It's reported, not applied: the raw model is already well calibrated, and applying it is a one-line decision if that changes.

Q73 How do you check fairness? **CORE**

The audit compares the model across `plan_type` groups (minimum 30 members each), on selection rate (who gets flagged) and on performance (ROC-AUC, recall). A ratio below 0.8 between worst and best group is flagged: the "four-fifths rule", a convention from US employment law.

Q74 Your fairness audit fails. What does that mean? **CORE** **TRICKY**

On the standard plan (949 subscribers in the test set) the model's ROC-AUC is 0.54, close to random, far below the other plans. So it's measurably worse at finding churners on that plan. The likely cause is too few standard-plan subscribers to learn from. It's written to the report, shown in Grafana, and fires the FairnessDisparity alert. Reported and alerted, not hidden, and not yet fixed.

If they push: "How would you fix it?" More standard-plan data, a per-plan threshold, or a model that learns plan-specific patterns. Then the gate and the audit would confirm it on the next run.

Q75 Why audit fairness by plan and not by age or gender?

Plan type is a model input and a business segment, and every subscriber has one. The members file has age and gender, but they're not model inputs and the KKBox values are often missing or implausible (age 0, for example). Auditing on protected attributes would need cleaner data; it's the right extension where it's available.

Q76 What is the capacity constraint?

A real retention team can only contact so many people. If the cost-optimal policy needs more offers than the budget allows, the model contacts the top-k by risk and the report prices the subscribers left uncovered. No cap is set for KKBox because the cost-optimal policy only needs 8 offers.

SECTION G

Explainability

Q77 What is SHAP? CORE

SHapley Additive exPlanations: a method from cooperative game theory for splitting a prediction among the features. Each feature's SHAP value is its average marginal contribution across all possible orderings of the features. For tree models, TreeExplainer computes them exactly and quickly.

Q78 Why is additivity important? CORE

Because it guarantees the explanation matches the prediction: base value plus every contribution equals exactly what the model predicted (in log-odds). So the reasons shown can't contradict the score. A test checks this for every row. Rule-based explanations don't have this property; they describe the input without knowing what the model did.

Q79 Why are SHAP values in log-odds?

Gradient boosting for classification works internally in log-odds and converts to a probability at the end with the logistic function. Additivity holds in log-odds; in probability space contributions don't add up simply. The dashboard shows the contribution sizes and the final probability.

Q80 How do you turn SHAP values into a sentence?

Contributions for related columns are summed into ten concepts (for example, `tenure_days` and `tenure_months` become "tenure"). That's valid because SHAP values add. Concepts are ranked by absolute size, and the top ones are phrased from the actual input: "relies on discounts (2 in the last 6 months)".

Q81 Feature importance vs SHAP: what's the difference? CORE

Feature importance is *global*: how much a feature matters across the whole dataset, with no direction. SHAP is *local*: how much each feature pushed *this* prediction, and in which direction. A retention agent needs the local view: why this subscriber.

Q82 What happens if SHAP fails?

The prediction still goes out, with a rule-based explanation and `explanation_method: "rules"`, so the client knows which kind it got. An explanation is a feature of a prediction, never a precondition for one.

Q83 What are the counterfactual levers on the dashboard? CORE

SHAP says what the model leaned on, not what a team could change. So for each lever the dashboard sends a second real request with one field changed and shows the new probability. For the at-risk example, turning auto-renew off takes 60% to 0.24%, and stopping discounts takes it to 0.79%. They're the model's actual answers, not inferences.

Q84 Should the business turn off auto-renew for everyone, then? TRICKY

No. That's exactly the trap. The model learned that auto-renew is *associated* with churn in KKBox, because it goes with cheap short plans. Switching it off wouldn't change who those subscribers are. A counterfactual shows what the model would predict, not what would happen in the world. Acting on it would need an experiment.

Q85 Tell me about the explanation bug you fixed.

A subscriber with auto-renew *off* was explained as "auto-renew enabled". The wording followed the direction of the SHAP effect rather than the input's value. For a number that's harmless ("inactive for 45 days" and "last seen 45 days ago" are both true), but for a yes/no field it states something false. It now states the actual value, with a test.

SECTION H

Registry, promotion and shadow scoring

Q86 What does MLflow do in your project? CORE

Two things. **Tracking** records every training run: parameters, metrics, the model file. The **registry** stores models as numbered versions under one name, with aliases (`@champion` , `@challenger`) that say which version is live.

Q87 What is a model registry, and why do you need one? CORE

A catalogue of trained models with versions and metadata. It gives one authoritative answer to "which model is in production, and how was it made?", and it makes promotion and rollback an explicit, recorded action instead of copying files around.

Q88 How does promotion work? CORE

A new model is registered as `@challenger` . The gate scores it and the current champion on the *same* held-out data with PR-AUC. If the challenger wins by at least 0.005, the `@champion` alias moves to it. If there's no champion yet, the first model is promoted unopposed. Every decision is recorded.

Q89 Why a margin of 0.005 instead of "any improvement"? CORE

Two equivalent models differ by random noise. With zero margin the gate would promote on noise about half the time, and the champion would flip back and forth forever. A margin means only a real improvement is enough to move production.

Q90 Why compare both models on the same data? TRICKY

Comparing a challenger's fresh score with a champion's score from an old run compares two different datasets. The difference could be the data, not the model. That's a reliable way to promote a worse model. Scoring both on the same rows in the same process makes the comparison fair.

Q91 How do you roll back a bad model? CORE

Move the `@champion` alias back to the previous version. Serving loads whatever the alias points to, so nothing is rebuilt or redeployed.

Q92 Why aliases rather than MLflow stages like "Production"?

MLflow 3 removed stages; aliases replace them. They're also more flexible: any number of named pointers (champion, challenger) rather than a fixed set of stages.

Q93 How do you know the gate works? CORE

CI proves it. The pipeline job trains twice on identical data; the second model is identical to the champion, so it *must* be rejected, and the job fails if it isn't. A rejected challenger is the system working correctly, so it's an info alert, never a page.

Q94 What is shadow scoring? CORE

When a distinct challenger exists, every live request is scored by both models, but only the champion's answer is returned. The challenger's answers are compared: agreement rate, how the number of flagged subscribers would change, probability divergence, errors and latency. It judges the challenger on real traffic with zero risk to users.

Q95 Shadow scoring vs A/B testing? TRICKY

In A/B testing, a share of real users actually receive the new model's decisions, so you can measure real outcomes (did they churn?) but users bear the risk. In shadow scoring nobody receives the challenger's decisions: zero risk, but you only learn how it *would* behave, not the outcomes. Shadow first, then an A/B test, is a common, safe sequence.

Q96 What if the challenger crashes during shadow scoring?

The champion's answer is returned regardless, and the error is counted. A rising count fires the ShadowChallengerErroring alert, which is exactly what should block that challenger's promotion.

Q97 Why does the dashboard show shadow scoring as "IDLE"?

Because there's currently no distinct challenger: the only registered model is the champion. It's built and tested (CI trains a distinct challenger and checks shadow scoring works); in the demo it honestly reports "no challenger" rather than a misleading 0%.

SECTION I

Serving and the API

Q98 Why FastAPI? CORE

Typed request validation through Pydantic, automatic interactive documentation at `/docs`, high performance, and a small amount of code. Invalid input is rejected with a clear 422 error before it reaches the model.

Q99 Walk me through a prediction request. CORE

`POST /predict` with nine fields. Pydantic validates types, ranges and that last activity isn't longer ago than tenure. The features go through the saved pipeline; the champion returns a probability; it's mapped to low, medium or high using the threshold. SHAP computes contributions and a sentence is built. If a challenger exists it scores in shadow. Metrics update, and the response returns the probability, label, band, threshold, explanation, attributions and method.

Q100 Difference between `/health` and `/ready`? CORE

`/health` is liveness: is the process running? If not, restart it. `/ready` is readiness: can it serve, meaning is a model loaded? If not, stop sending it traffic, but don't kill it. Kubernetes uses these for its liveness and readiness probes.

Q101 What happens if no model is loaded?

The service still starts and answers `/health`; `/ready` reports not ready, and `/predict` returns 503 Service Unavailable with a clear message rather than a 500 crash. The `ModelNotLoaded` alert fires after five minutes.

Q102 How are the risk bands defined?

Low: below the threshold (5%). High: above the point halfway from the threshold to 100%, about 52.5%. Medium: in between.

Q103 How does the API know which model to load?

The setting `SDD_MODEL_SOURCE`: `registry` loads the `@champion` from MLflow; `local` loads the saved artifact; `auto` tries the registry first and falls back to local. In the Docker stack, the promoted artifact is mounted read-only and loaded locally.

Q104 Why is there a batch endpoint?

Scoring many subscribers in one call (1 to 1,000) avoids per-request overhead: one validation pass, one model call. It's how a nightly list or the stream scorer scores efficiently.

Q105 How would this scale to many requests?

The API is stateless apart from in-memory metrics, so it scales horizontally: more replicas behind a load balancer. The Kubernetes manifests do exactly this with an autoscaler from 2 to 10 replicas at 70% CPU. Prediction is fast (milliseconds); SHAP is the heavier part, and could be made optional per request if needed.

Q106 Is the API secure? **TRICKY**

It validates all input and runs as a non-root user in the container, but it has **no authentication**: it's designed for a trusted internal network. For real use it would sit behind an API gateway with authentication, rate limiting and HTTPS. No secrets are stored in the repository, and a test fails the build if credentials appear in the alerting config.

Q107 How did you build and test the dashboard?

It's a single self-contained HTML page served by the API: no build step, no external libraries. Tests check its structure against the API schema, and it was verified by rendering it in headless Chrome and inspecting screenshots. That caught problems the structural tests missed, including a version that rendered completely blank.

SECTION J

Streaming

Q108 Why have streaming if you already have an API? CORE

The API answers when asked. Subscriber events happen continuously, and the business wants a fresh risk score when something changes, not only when someone thinks to ask. A stream scorer reacts to events as they arrive and publishes scores for other systems to consume.

Q109 What is Kafka? What is Redpanda? CORE

Kafka is a distributed log: producers append messages to *topics*, and consumers read them in order at their own pace, tracking their position with *offsets*. Redpanda is a Kafka-compatible broker: the same protocol, as a single binary without the JVM or ZooKeeper, which is light enough for a laptop and CI. The code uses a standard Kafka client and would run unchanged against Apache Kafka.

Q110 Walk me through the streaming loop. CORE

Poll up to 200 messages from `subscriber-events`; validate each; score the valid ones in one batch with the same code as the API; write scores to `subscriber-scores`, keyed by subscriber; send invalid ones to `subscriber-scores-dlq` with a reason; wait for the broker to confirm the writes; only then commit the input offsets.

Q111 What delivery guarantee do you provide? CORE

At-least-once. Offsets are committed only after the scores are durably written (`acks=all`). If the scorer crashes after writing but before committing, it re-reads and re-scores those messages: a duplicate, never a loss. Auto-commit is off, because it could acknowledge a message whose score was never written.

Q112 Why not exactly-once? TRICKY

Exactly-once needs Kafka transactions and more complexity. Here duplicates are harmless: scoring the same subscriber twice gives the same score, so the operation is effectively idempotent. At-least-once plus idempotent processing gives the practical effect of exactly-once, far more simply.

Q113 What is a dead-letter queue, and why do you need one? CORE

A separate topic for messages that can't be processed. Without one, a malformed "poison" message crashes the consumer; on restart it reads the same message and crashes again, forever. With it, the bad message is set aside with its reason and original payload, the batch continues, and someone can inspect it later. CI sends poison messages to prove this.

Q114 If the model is unavailable, do messages go to the dead-letter queue? TRICKY

No, deliberately. That would throw away perfectly good data because of a temporary problem. Instead the batch fails and offsets aren't committed, so the messages are retried once the model is back. The DLQ is only for messages that are themselves bad.

Q115 Why key the output by subscriber?

Kafka keeps order only within a partition, and messages with the same key go to the same partition. Keying by subscriber keeps each person's scores in order, so a consumer never sees an older score arrive after a newer one.

Q116 How would you scale the stream scorer?

Run more instances in the same consumer group: Kafka splits the topic's partitions between them. The Kubernetes manifest runs 2 replicas. The limit is the number of partitions, since each is read by one consumer in the group.

Q117 How do you monitor the stream scorer?

It exposes its own Prometheus endpoint on port 8001, scraped as a separate job, so a dead consumer isn't hidden behind a healthy API. The StreamScorerDown alert fires if it stops answering, and throughput is graphed separately from the API's.

SECTION K

Monitoring, drift and alerting

Q118 Why do ML models need special monitoring? CORE

Ordinary software fails loudly: errors, crashes. A model fails *quietly*: it keeps returning confident predictions while the world changes, and nothing errors. Monitoring the inputs, the outputs and, eventually, outcomes is how that silent decay becomes visible.

Q119 What is data drift? Concept drift? CORE

Data drift (covariate shift): the distribution of the inputs changes, for example people listen less.

Concept drift: the relationship between inputs and outcome changes, for example the same behaviour now means something different because a competitor appeared. Data drift is detectable immediately from inputs alone; concept drift needs outcomes (labels), which arrive later.

Q120 Which does your system detect? TRICKY

Data drift, per feature, plus drift in the model's output distribution, using PSI. Concept drift needs true labels, which only arrive 30 days later. The scheduled pipeline covers it indirectly: each retrain is evaluated on fresh labelled data and the gate compares it with the champion. Live accuracy monitoring from delayed labels would be the next step.

Q121 Explain PSI. CORE

The Population Stability Index compares two distributions of one feature. Split the training data into 10 quantile bins; put live data into the same bins; for each bin compare the live share a with the training share e ; sum $(a - e) \times \ln(a / e)$. Identical distributions give 0. Below 0.1 is stable, 0.1–0.25 moderate, above 0.25 significant.

Q122 Why PSI and not a statistical test like Kolmogorov–Smirnov? TRICKY

With large samples, tests like KS flag tiny, meaningless differences as "significant", because the p-value shrinks with sample size. PSI measures the *size* of the shift on a stable scale, with industry-standard thresholds, which is what matters for deciding whether to retrain. It's also easy to compute from stored bin proportions.

Q123 What did drift detection find on real data? CORE

Run on February 2017, the latest real month, read from Postgres, against the training months: 7,614 subscribers, verdict **stable**, every feature's PSI below 0.06. KKBox's population didn't shift in that period. That's a real result, not a failure to detect: the dashboard's drift test proves the detector fires when something changes.

Q124 How do you know the drift detector works? CORE

By testing it on a change I control. The dashboard creates 400 subscribers, then a copy with sessions cut to near zero and last activity pushed back 45 days. The detector ranks exactly those two features at the top, with the other seven under 0.5. CI does the same with injected drift, which must be detected.

Q125 What is the reference profile?

A snapshot of the training distribution, saved when the model is trained: bin edges and proportions for each feature and for the predictions. Drift is always measured against the profile of the model that's currently serving, not against whatever data is newest.

Q126 Why Prometheus? CORE

It's the standard open-source metrics system: a time-series database, a query language (PromQL), and alert rules evaluated alongside the data. It *pulls* metrics from services every 15 seconds, so a service that stops answering is itself a signal.

Q127 Pull vs push monitoring?

In pull, the monitor scrapes each service; the monitor controls timing, and a silent service is immediately visible. In push, services send metrics; that suits short-lived jobs that may finish before a scrape. Prometheus is pull-based; the batch pipeline writes a report file that the API exposes, rather than pushing.

Q128 Counter, gauge, histogram: what's the difference? CORE

A **counter** only goes up (predictions served); you graph its rate. A **gauge** goes up and down (model loaded, flagged rate). A **histogram** counts observations into buckets (prediction probabilities), so percentiles like p90 can be computed.

Q129 How did you decide what to alert on? CORE

By asking whether a human would act on it. "Drift is significant" is actionable: someone decides whether to retrain. "PSI moved from 0.04 to 0.06" isn't. Paging people for trivia teaches them to ignore alerts. A rejected challenger is the gate working, so it's informational, never a page.

Q130 What does Alertmanager do?

It receives firing alerts from Prometheus, groups related ones into one notification, routes them by severity (critical to on-call immediately, warning for review, info recorded), and repeats them at sensible intervals. It also **inhibits**: if the API is down, the warnings that are merely symptoms of that are suppressed.

Q131 What is alert inhibition, with an example?

Suppressing an alert that another alert already explains. If ApiDown is firing, a FlaggedRateSpike warning for the same component is noise: the real problem is the outage. Inhibition shows only the cause, not every symptom.

Q132 Is any alert firing right now? TRICKY

Yes: FairnessDisparity, and it's correct. The model's fairness audit genuinely fails on the standard plan. Earlier, two false alerts (PipelineStale and ModelNotLoaded) were firing because the stream scorer exported default values for the API's metrics. I found that by looking at Grafana, and fixed it by scoping every API-state query to the API's job, with a test.

Q133 What does the Grafana dashboard show?

Six sections: whether the model is up and what it's serving; prediction traffic and score distribution; drift and the retraining pipeline; decision quality (fairness, calibration, savings); shadow scoring; and outreach capacity. It's provisioned from a JSON file, so it's version-controlled like code.

Q134 Why does Grafana show "NOT RUN" for last promotion?

Because the most recent pipeline run was the monitoring-only flow, which checks drift and trains nothing. It neither promoted nor rejected anything. Earlier the panel said "REJECTED", because a gauge's default value 0 means rejected. That was a false claim, so those gauges now start as "no value".

SECTION L

Orchestration and reproducibility

Q135 What does Prefect do? CORE

It runs the pipeline (ingest, drift check, train and gate, report) as a flow of tasks, with retries, structured logs and a run history, and it can serve the flow on a schedule (nightly at 03:00 by default). There's also a monitoring-only flow and a backfill flow.

Q136 Why Prefect rather than cron or Airflow?

Cron runs a command but gives no retries, dependencies or history. Airflow is powerful but heavy: a scheduler, a database and a webserver to operate. Prefect turns ordinary Python functions into tasks with a decorator and runs locally with no infrastructure, which suits a project this size. The steps are plain functions, so moving to Airflow later wouldn't mean rewriting them.

Q137 Why is drift checked before retraining? CORE

Training writes a new reference profile from the latest data. Checking drift afterwards would compare the data with a profile built from itself, so it would always say "stable" and never detect anything. Checked before, it asks whether the world has moved since the *serving* model was fitted.

Q138 What does idempotent mean, and why does it matter? CORE

Running something twice gives the same result as running it once. If the nightly job fails halfway and is re-run, it mustn't double every subscriber's history or promote twice. CI runs the pipeline twice and checks the second run changes nothing.

Q139 What is DVC, and what do you use it for? CORE

Data Version Control. `dvc.yaml` declares pipeline stages (simulate, train) with their commands, input files, parameters and outputs. `dvc repro` re-runs only stages whose inputs changed, and `dvc.lock` records the exact hash of every input and output. Any result can be traced to exact data, code and parameters.

Q140 How would someone reproduce your model? CORE

Check out the commit, load the same data, and run the training with the same `params.yaml`. The seed is fixed and the split is by date, so the result is deterministic. The model's metadata records its parameters, split sizes, training time and library versions, and `constraints-model.txt` pins those libraries.

Q141 Why keep settings in params.yaml?

So every tunable number (windows, split, hyperparameters, costs, drift and fairness thresholds, the promotion margin) lives in one reviewable file rather than scattered through code. Changing the model is a one-line, versioned change. Any value can also be overridden with an `SDD_*` environment variable.

Q142 What does "needs attention" mean in a pipeline run?

A human should look: a challenger was rejected, drift was significant, or fairness failed. None of these fails the pipeline: a rejected challenger means the gate worked, and drift means the world moved. Failing the run would train people to ignore failures. So it's an info alert and a note in the report.

SECTION M

CI/CD and testing

Q143 What is CI/CD? CORE

Continuous integration: every change is automatically built and tested, so problems surface immediately. **Continuous delivery:** every change that passes is packaged into a releasable artifact. **Continuous deployment:** that artifact is also released to production automatically.

Q144 What does your CI do? CORE

Seven jobs on every push: lint and 434 tests; train and gate on SQLite; train and gate on Postgres plus the demo queries; the full pipeline (idempotent, identical challenger rejected, drift detected); the streaming loop with poison messages; building and smoke-testing the Docker image; and, on `main`, publishing the image to the GitHub Container Registry.

Q145 Do you have CD? TRICKY

Continuous *delivery*, yes: every merge to `main` that passes all six test jobs publishes the image, tagged with the commit hash. Continuous *deployment*, no: nothing automatically releases it to a production cluster, because there's no free cluster that fits the stack. I'd describe it exactly that way.

Q146 Why tag the image with the commit hash?

So any running deployment can be traced to exactly the code that built it, and rolled back by deploying the previous hash. `latest` alone says nothing about what's inside.

Q147 Tell me about CI being broken for 17 days. CORE TRICKY

A test required that every file a DVC stage depends on exists on disk. One dependency, the warehouse database, is git-ignored and produced by a pipeline stage: present on my machine, absent on every fresh checkout, which is what CI is. So CI failed on every push while local tests passed, and because that job gates the others, four jobs didn't run at all. I found it by running the suite on a fresh clone. The rule is now "in the repo or produced by a stage", and the demo checklist checks CI is green.

Q148 How many tests are there, and what do they cover?

434, across 15 files: monitoring, evaluation, streaming, the API, Prometheus and alerting config, the KKBox loader, point-in-time features, orchestration, training, the registry, shadow scoring, explanations, features, DVC and the Postgres copy.

Q149 Which test is most important? CORE

The leakage test: adding events after the cutoff must not change any feature. If it failed, every metric the project reports would be meaningless.

Q150 What kinds of tests do you have?

Unit tests (PSI maths, cost formulas); integration tests (training end to end on a small warehouse, API requests); property tests (SHAP contributions add up for every row); configuration tests (every alert references a real metric, no credentials, no overlapping dashboard panels); and system tests in CI (idempotency, the gate, drift, poison messages).

Q151 How do you test something that needs Postgres or Kafka?

In CI, Postgres runs as a GitHub Actions service container, and the full train-and-gate run executes against it. For Kafka, the stream logic is tested against an in-memory broker with the same interface, so the loop, dead-lettering and commit ordering are tested without a real broker.

Q152 What is linting?

Automatic checking of code for errors and style problems without running it. The project uses ruff; CI fails on any finding.

SECTION N

Docker, Compose and Kubernetes

Q153 What is Docker, and why use it? CORE

Docker packages an application with everything it needs (runtime, libraries, configuration) into an image that runs identically on any machine as a container. It removes "works on my machine" problems and lets eight services start with one command.

Q154 Container vs virtual machine?

A VM emulates a whole computer with its own operating system: heavy, slow to start. A container shares the host's kernel and isolates only the application's processes and files: light, starting in seconds. Many containers fit where a few VMs would.

Q155 What is Docker Compose? CORE

A tool that defines a multi-container application in one file (services, images, ports, volumes, environment, dependencies) and starts it all with one command. This project's `docker-compose.yml` defines 8 services.

Q156 What is a Docker volume, and why use one for Postgres?

Storage managed by Docker that lives outside any container. Containers are disposable; if the database's files were inside the container, recreating it would delete the data. The 82.8 million rows live in a named volume, so they survive restarts and upgrades.

Q157 What's special about your Dockerfile?

It's multi-stage: a builder stage installs dependencies and trains, and a slim runtime stage copies only what's needed, so the image is smaller with less attack surface. The service runs as a non-root user, has a health check, and installs the model's libraries pinned to the training versions.

Q158 Why mount the model into the container instead of building it in? TRICKY

The real model file is git-ignored, and the image must build from a clean checkout, so the image trains a synthetic stand-in. Mounting the registry-promoted artifact read-only means the container serves exactly the champion. Before that fix, the stack silently served the synthetic model. The demo checklist now verifies the served model's training date.

Q159 Why pin library versions for the model?

A pickled scikit-learn model should be loaded by the same version that trained it; a different version can load it without complaint and score differently. One rebuild quietly moved scikit-learn from 1.9.0 to 1.9.1. Now `constraints-model.txt` pins them, and the checklist compares the container's versions with the model's metadata.

Q160 What is Kubernetes, and how does your project use it? **CORE**

A system for running containers across a cluster of machines: it keeps the requested number of copies running, restarts failures, scales, and rolls out updates. The project has manifests for the API and stream scorer: Deployments with 2 replicas, resource limits, health probes, an autoscaler (2–10 replicas at 70% CPU), a PodDisruptionBudget and a ConfigMap. They're written but not deployed, since no free cluster fits the stack.

Q161 What are liveness, readiness and startup probes?

Liveness: if it fails, Kubernetes restarts the container. **Readiness:** if it fails, traffic stops being sent to it (without a restart). **Startup:** gives a slow-starting container time, so a long model load isn't mistaken for a crash. Here they use `/health`, `/ready` and `/health` respectively.

Q162 What is a HorizontalPodAutoscaler? A PodDisruptionBudget?

An HPA adds or removes replicas to keep a metric (here CPU at 70%) near its target, between a minimum and maximum (2 and 10). A PDB limits voluntary disruptions, such as node maintenance, so at least a set number of pods (here 1) stays running.

Q163 Why isn't it deployed to the cloud? **TRICKY**

Cost and size. The full stack (a 15 GB warehouse, MLflow, Kafka, Prometheus, Grafana) doesn't fit any free tier, and the project was built on free resources only. It runs in Docker locally, the image is published on every merge, and the Kubernetes manifests are ready. Deploying to a managed cluster is the stated next step.

SECTION 0

General ML and MLOps theory

Q164 Supervised vs unsupervised learning? CORE

Supervised learning trains on examples with known answers (labels) to predict answers for new cases; churn prediction is supervised classification. Unsupervised learning finds structure without labels, such as clustering subscribers into segments.

Q165 Classification vs regression?

Classification predicts a category (churn: yes or no); regression predicts a number. This model is a classifier that outputs a probability, which a threshold turns into a decision.

Q166 What is the bias–variance trade-off? CORE

Bias is error from a model too simple to capture the pattern (underfitting). Variance is error from a model so flexible it fits noise (overfitting). Reducing one tends to increase the other. Boosting reduces bias with many shallow trees; its learning rate, depth and subsampling keep variance in check.

Q167 What is regularisation?

Anything that discourages a model from fitting noise. In gradient boosting: a small learning rate, shallow trees, a minimum number of samples per leaf, and subsampling, all used here.

Q168 How does a decision tree decide where to split?

It tries candidate splits on each feature and picks the one that most reduces impurity (or, in gradient boosting, most reduces the loss) in the resulting groups. It repeats on each group until a stopping rule applies, such as maximum depth or minimum leaf size.

Q169 What is one-hot encoding?

Turning a category into separate yes/no columns, one per value. Plan type becomes `plan_type_basic`, `plan_type_standard` and `plan_type_premium`. It stops a model treating categories as if they had an order.

Q170 What is the logistic (sigmoid) function?

It maps any number to a probability between 0 and 1: $1 \div (1 + e^{-x})$. Gradient boosting for classification adds up tree outputs as log-odds, then applies it to get the probability.

Q171 What is model decay? CORE

A deployed model getting worse over time because the world it learned from changes: data drift or concept drift. It's the reason for monitoring and scheduled retraining.

Q172 What is a feature store, and would you need one?

A system that computes and stores features once and serves them consistently to both training (history) and serving (latest values), preventing training–serving skew. This project gets the same guarantee more simply: one saved pipeline, and point-in-time SQL over the warehouse. At larger scale, with many models sharing features, a feature store would be worth it.

Q173 What are the MLOps maturity levels?

A common framing (from Google): level 0 is manual; a data scientist trains and hands over a model. Level 1 automates the training pipeline, with continuous training and monitoring. Level 2 adds CI/CD for the pipeline itself. This project has automated, gated training, monitoring and CI that tests the pipeline and publishes the service. It's around level 1, reaching toward 2, without automatic production deployment.

Q174 Batch vs online (real-time) inference?

Batch scores many records on a schedule (a nightly list); online scores one request on demand with low latency. This system does both: batch through `/predict/batch` and the pipeline, online through `/predict`, plus streaming, which is continuous scoring as events arrive.

Q175 What is data versioning, and why does it matter?

Tracking exactly which data a model was trained on, like Git tracks code. Without it, you can't reproduce a model, explain why it changed, or roll back to the data behind a good version. DVC does this here.

Q176 What is a champion–challenger setup?

The current production model (champion) keeps serving while a candidate (challenger) is evaluated against it, on held-out data and in shadow on live traffic. The challenger replaces it only on evidence. It's exactly what the MLflow aliases and the gate implement.

Q177 Correlation vs causation, in your project? CORE

The model learns correlations. Auto-renew being associated with churn doesn't mean auto-renew *causes* churn; it goes with cheap short plans. Acting on a model's features as if they were causes needs an experiment, such as an A/B test of an intervention.

SECTION P

Limitations, ethics and hard questions

Q178 What are the biggest weaknesses of your project? CORE

The model is modest (PR-AUC 0.073, recall 16.8%) because the data lacks strong signals. The fairness audit fails on the standard plan. The cost figures are placeholders. The churn label is stricter than Kaggle's. And it isn't deployed to a public cloud: continuous delivery, not deployment.

Q179 Would you deploy this model to production? TRICKY

The system, yes; this model, only with care. It's far better than random and well calibrated, so it can usefully rank who to look at first. But it misses most churners, and the fairness failure should be understood before it drives decisions about the standard plan. I'd use it to prioritise human attention, not to act automatically.

Q180 Are there ethical concerns with churn prediction? CORE

Yes. It's behavioural data about people, so privacy and data protection apply. Offers could be allocated unfairly across groups, which is why there's a fairness audit. And the model could be misused, for example to make cancelling harder for people flagged as at risk. It should be used to improve retention offers, never to obstruct leaving.

Q181 How did you verify your results are real? CORE

The leakage test; a time-based split; cross-checking the feature pipeline on two databases row for row; checking the running system's served model and preset scores with a 19-point checklist; screenshots of every screen; and running the test suite on a fresh clone, as CI does.

Q182 What would you do differently if you started again?

Test against Postgres and a fresh checkout from day one. SQLite and my own working directory hid several bugs. And look at the real outputs (dashboards, containers, CI) as a habit, not only when something seems off.

Q183 How did you build this? Did you use AI tools? CORE TRICKY

Answer truthfully. Check your college's policy on AI assistance beforehand. If you used an AI coding assistant, say so plainly and explain what *you* did: chose the problem and data, made or approved the design decisions, ran and verified everything on your own machine, and understood the system well enough to answer every question in this book. Being open about the tools while clearly understanding the system is far stronger than a claim that falls apart under one follow-up.

Q184 Which part are you most proud of?

A good answer names one thing and why. For example: the point-in-time warehouse, because it's what makes every metric trustworthy. Or the PR-AUC gate, because the real data proved the choice right. Or finding the false alerts, because it showed that a green test suite isn't the same as a correct system.

Q185 Can you show me it working right now? CORE

Yes. `make demo` starts all eight services and `make demo-check` verifies 19 things. Then: the dashboard at port 8000, the warehouse in TablePlus, MLflow at 5050, Grafana at 3000, the alerts at 9090, and the green CI runs on GitHub. If anything fails live, the screenshots in `docs/viva/screenshots/` are the fallback.

APPENDICES

Quick reference

The one-page summary, every number worth knowing, every command, and a glossary. The pages to reread the night before.

APPENDIX A

The whole project on one page

Problem	Predict which subscribers of a music-streaming service cancel in the next 30 days, explain why, and run it as a trustworthy production system.
Data	KKBox (WSDM Cup 2018, Kaggle): 31.3 GB, 6.77M subscribers. Cleaned into a Postgres event warehouse of 82.8M rows. 2.66M orphan rows dropped rather than invented.
No leakage	Features from the 30 days before a cutoff; label from the 30 days after; windows never overlap; split by time (train Nov–Dec 2016, test Jan 2017).
Model	Gradient boosting, 300 trees of depth 3. Test PR-AUC 0.073 (random 0.015: 4.8× better), ROC-AUC 0.835, recall 16.8% at threshold 0.05, calibration error 0.004.
Key finding	On real data ROC-AUC rose (0.674 → 0.835) while PR-AUC fell (0.345 → 0.073). So the gate uses PR-AUC.
Audit	Calibrated; the cost audit prefers threshold 0.43 under placeholder costs; the fairness audit fails on the standard plan (reported and alerted).
Registry	MLflow. Live model = @champion . A challenger is promoted only if it beats the champion on PR-AUC, on the same data, by ≥ 0.005 . Rollback = move the alias back.
Serving	FastAPI with SHAP explanations that add up exactly to the prediction; a dashboard with counterfactual levers; a Kafka stream scorer, at-least-once, with a dead-letter queue.
Monitoring	Prometheus (15 alert rules), Alertmanager (routing, inhibition), Grafana. PSI drift on real Feb 2017 data: stable, every feature under 0.06.
Automation	Prefect pipeline (ingest → drift → train and gate → report), idempotent. DVC + params.yaml. GitHub Actions: 7 jobs, 434 tests, image published to ghcr.io.
Deployment	8 Docker services, make demo + 19-point make demo-check . Kubernetes manifests written, not deployed.
Limitations	Modest model; fairness failure; placeholder costs; stricter churn label than Kaggle's; not on a public cloud.

APPENDIX B

Numbers worth knowing

Raw data	31.3 GB	Test PR-AUC	0.073
Subscribers	6,769,473	Test ROC-AUC	0.835
Subscription events	18,891,703	Validation PR-AUC / ROC-AUC	0.052 / 0.834
Payments	18,891,703	Threshold (best F1)	0.05
Sessions (listening days)	38,216,556	Recall · precision @ 0.05	16.8% · 7.6%
Warehouse total	82.8M rows · 16 GB	TP · FP · FN · TN	85 · 1,040 · 421 · 31,496
Raw listening log	392M rows	Calibration error (ECE)	0.004
Orphan rows dropped	2,656,043	Cost-optimal threshold	0.43 (saves £16,508)
Unknown subscribers	432,623 (18.3%)	Standard-plan ROC-AUC	0.54 (n = 949)
Postgres load time	~7.5 min	Promotion margin	0.005 PR-AUC
Feature query: SQLite vs Postgres	506 s vs 6.8 s	PSI stable / significant	< 0.10 / > 0.25
One subscriber's history	17 ms	Real drift (Feb 2017)	max PSI 0.058
Train / val / test rows	66,704 / 33,600 / 33,042	Alert rules	15
Churn rate (test)	1.53%	Tests · CI jobs	434 · 7
Churners in test	506	Docker services	8
Trees · depth · learning rate	300 · 3 · 0.05	Demo checks	19

The three demo presets: **at-risk 60.37% (high)**, **borderline 5.01% (medium)**, **healthy 0.05% (low)**. Turning auto-renew off takes the at-risk subscriber from 60% to 0.24%.

APPENDIX C

Commands

COMMAND	WHAT IT DOES
<code>make demo-prepare</code>	Build the image and write the drift report (once, before a demo)
<code>make demo</code>	Start everything, wait until healthy, warm up, print URLs
<code>make demo-check</code>	The 19-point pre-demo checklist
<code>make demo-down</code>	Stop everything, keep all data
<code>make warehouse-postgres</code>	Copy the SQLite warehouse into Postgres (82.8M rows)
<code>make monitor</code>	Drift check on Postgres, no retraining
<code>make pipeline</code>	The full retraining pipeline once
<code>make pipeline-drift</code>	The pipeline after injecting drift
<code>make train-promote</code>	Train, register in MLflow, run the gate
<code>make mlflow-ui</code>	Browse runs and the registry at port 5050
<code>make serve</code>	Run the API locally with auto-reload
<code>make stream-events</code> / <code>stream-poison</code>	Publish sample / malformed events to Kafka
<code>make test</code> · <code>make lint</code>	Run the tests · lint the code
<code>make repro</code> · <code>make metrics-diff</code>	Rerun changed DVC stages · compare metrics with main
<code>make help</code>	List every target

APPENDIX D

Glossary

A/B test

Real users are split between two versions and real outcomes compared.

Alias (MLflow)

A named pointer to a model version, such as `@champion`. Moving it promotes or rolls back.

Alertmanager

Groups, routes and suppresses alerts from Prometheus.

At-least-once delivery

Every message is processed one or more times; duplicates possible, loss not.

Baseline

The simplest reference a model must beat, such as random guessing.

Bias-variance trade-off

Too simple underfits (bias); too flexible overfits (variance).

Brier score

Mean squared error of predicted probabilities; lower is better.

Calibration

Whether predicted probabilities match observed frequencies.

Champion / challenger

The production model / the candidate evaluated against it.

Churn

A customer leaving. Here: an explicit cancellation within 30 days.

CI / CD

Continuous integration (build and test every change) / continuous delivery or deployment (release it).

Concept drift

The relationship between inputs and outcome changes.

Confusion matrix

Counts of true/false positives and negatives.

Container

An isolated, portable package of an application and its dependencies.

Counterfactual

"What would the model predict if this one input were different?"

Cutoff date

The "as of" date for a training example; features before it, the label after.

Data contract

Rules incoming data must satisfy before it's loaded.

Data drift

The distribution of inputs changes over time.

Data leakage

Training on information that wouldn't be available at prediction time.

Dead-letter queue

Where unprocessable messages go, with a reason, instead of crashing the consumer.

DVC

Data Version Control: versions data and pipeline stages.

ECE

Expected calibration error: the average gap between predicted and actual rates.

Event warehouse

A store of immutable, timestamped events, rebuildable as of any date.

F1

Harmonic mean of precision and recall.

Feature store

A system serving identical features to training and serving.

Four-fifths rule

A fairness convention: a group ratio below 0.8 is flagged.

Gauge / counter / histogram

Prometheus metric types: up-and-down value / only-increasing count / bucketed distribution.

Gradient boosting

Trees built one after another, each correcting the errors so far.

Grafana

Dashboards over Prometheus data.

HPA

HorizontalPodAutoscaler: Kubernetes adds or removes replicas to hold a target.

Idempotent

Running twice has the same effect as running once.

Inhibition

Suppressing alerts already explained by another alert.

Isotonic calibration

A monotonic mapping from raw scores to calibrated probabilities.

Kafka / Redpanda

A distributed log for event streams / a Kafka-compatible broker.

Liveness / readiness

Is the process up? / Can it serve traffic now?

Log-odds

$\ln(p \div (1 - p))$; the scale boosting and SHAP work in.

MLflow

Experiment tracking and model registry.

MLOps

Practices for building, deploying and maintaining ML systems reliably.

Offset (Kafka)

A consumer's position in a topic partition.

Overfitting

Learning noise; good on training data, poor on new data.

Point-in-time correctness

Features use only what was known at the cutoff.

Poison message

A message that crashes a consumer every time it's read.

PR-AUC

Area under the precision–recall curve; random equals the positive rate.

Precision / recall

Of those flagged, share correct / of true positives, share flagged.

Prefect

Workflow orchestration: tasks, retries, schedules.

Prometheus

Pull-based metrics database with alert rules.

PSI

Population Stability Index: $\sum (a - e) \ln(a/e)$ over bins.

Reference profile

The training distribution, saved for drift comparisons.

ROC-AUC

Probability a random positive is ranked above a random negative.

Shadow scoring

A challenger scores live traffic silently; only the champion's answer is used.

SHAP

Game-theoretic feature contributions that add up to the prediction.

Temporal split

Train on earlier periods, test on later ones.

Threshold

The probability above which a subscriber is flagged.

Training–serving skew

Features computed differently in training and in production.

Volume (Docker)

Storage that outlives any container.